Human

Human
Agent

Human
Agent

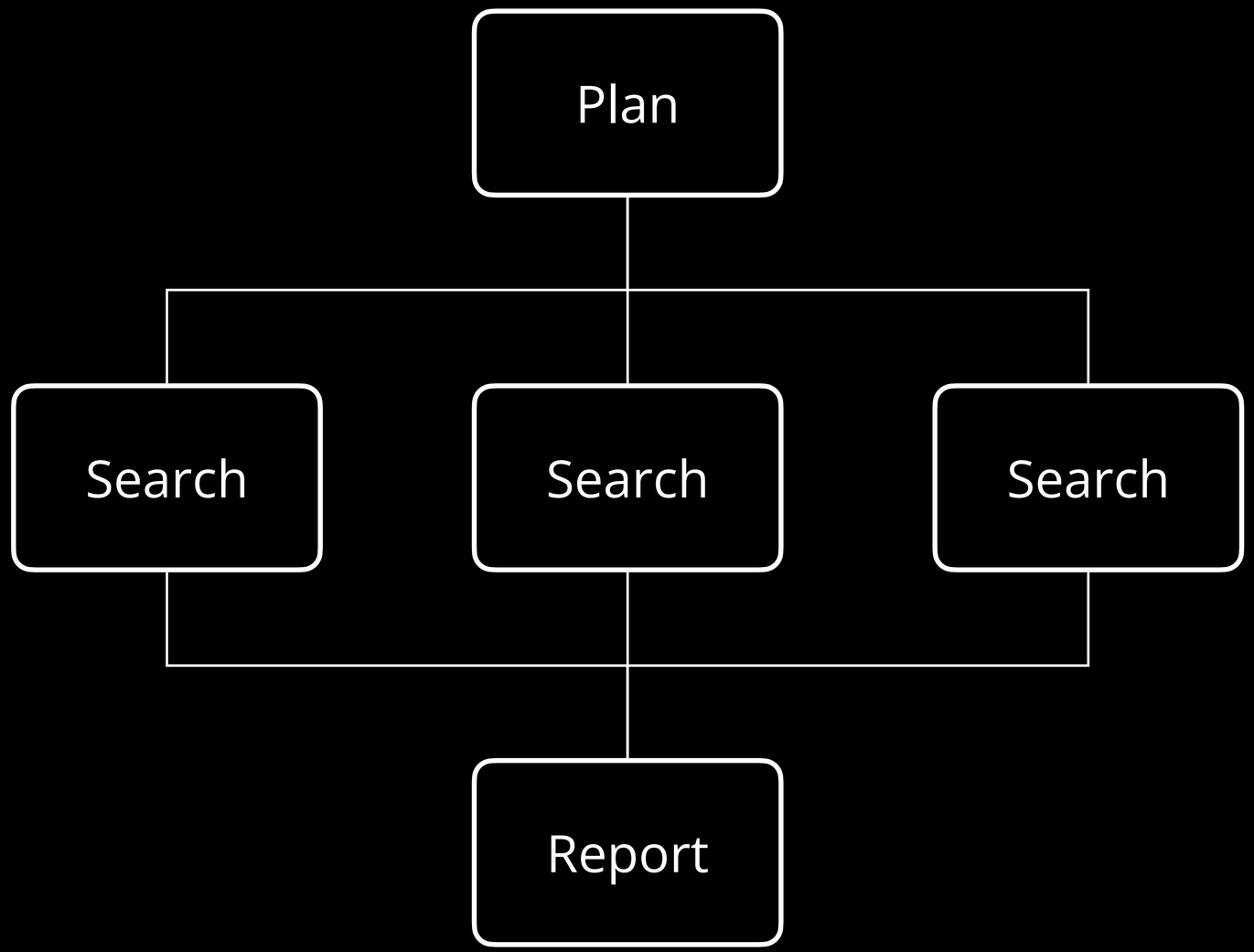
Agent
Human

Agent
Human

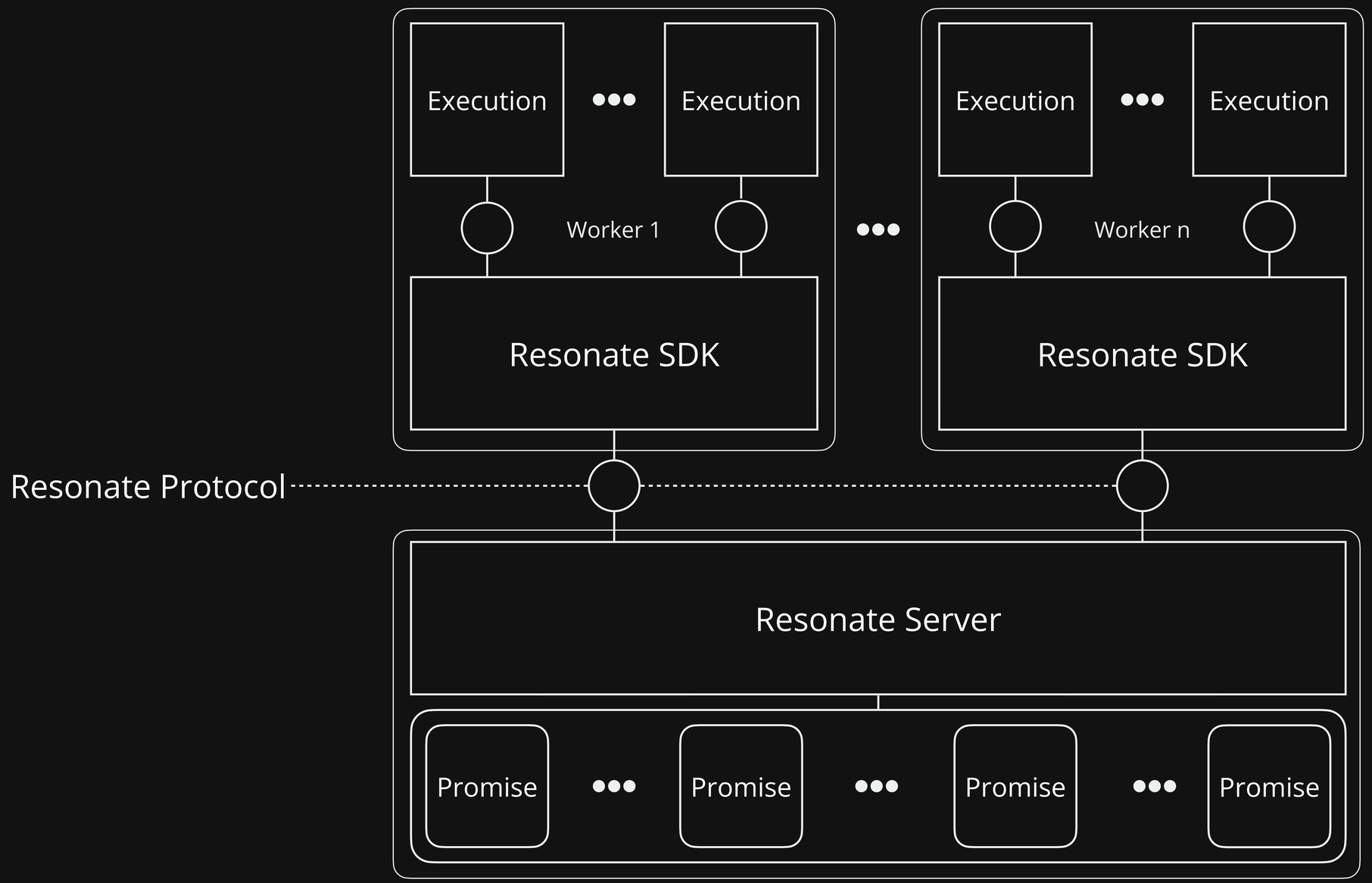
Agent

Dominik Tornow

Resonate HQ

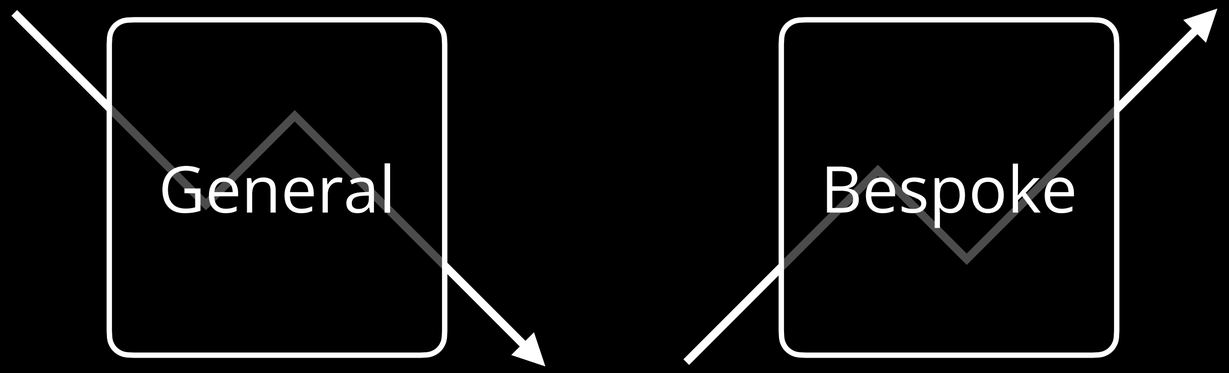

Plan
Search
Search
Search
Report

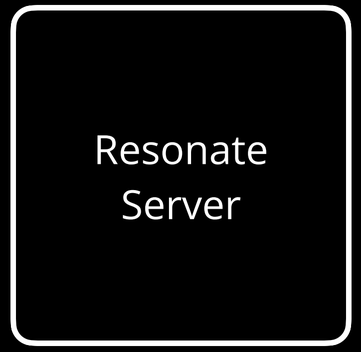
```
async function research(context: Context, question: string) {
  // Plan the searches
  const searches = await context.run(plan, question);
  // Fan out the searches
  const results = await Promise.all(
    searches.map(async (s) => {
      try {
        return await context.rpc<Result>("search", s.query);
      } catch {
        return { query: s.query, text: "" };
      }
    })
  );
  // Synthesize the results
  return await context.run(report, question, results);
}
```

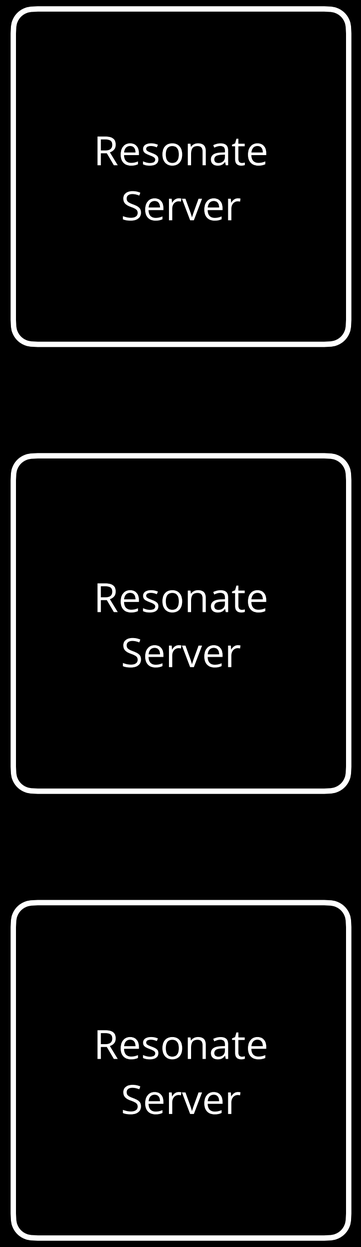

Execution
Execution
Execution
Execution
Worker 1
Worker n
Resonate SDK
Resonate SDK
Resonate Protocol
Resonate Server
Promise
Promise
Promise
Promise

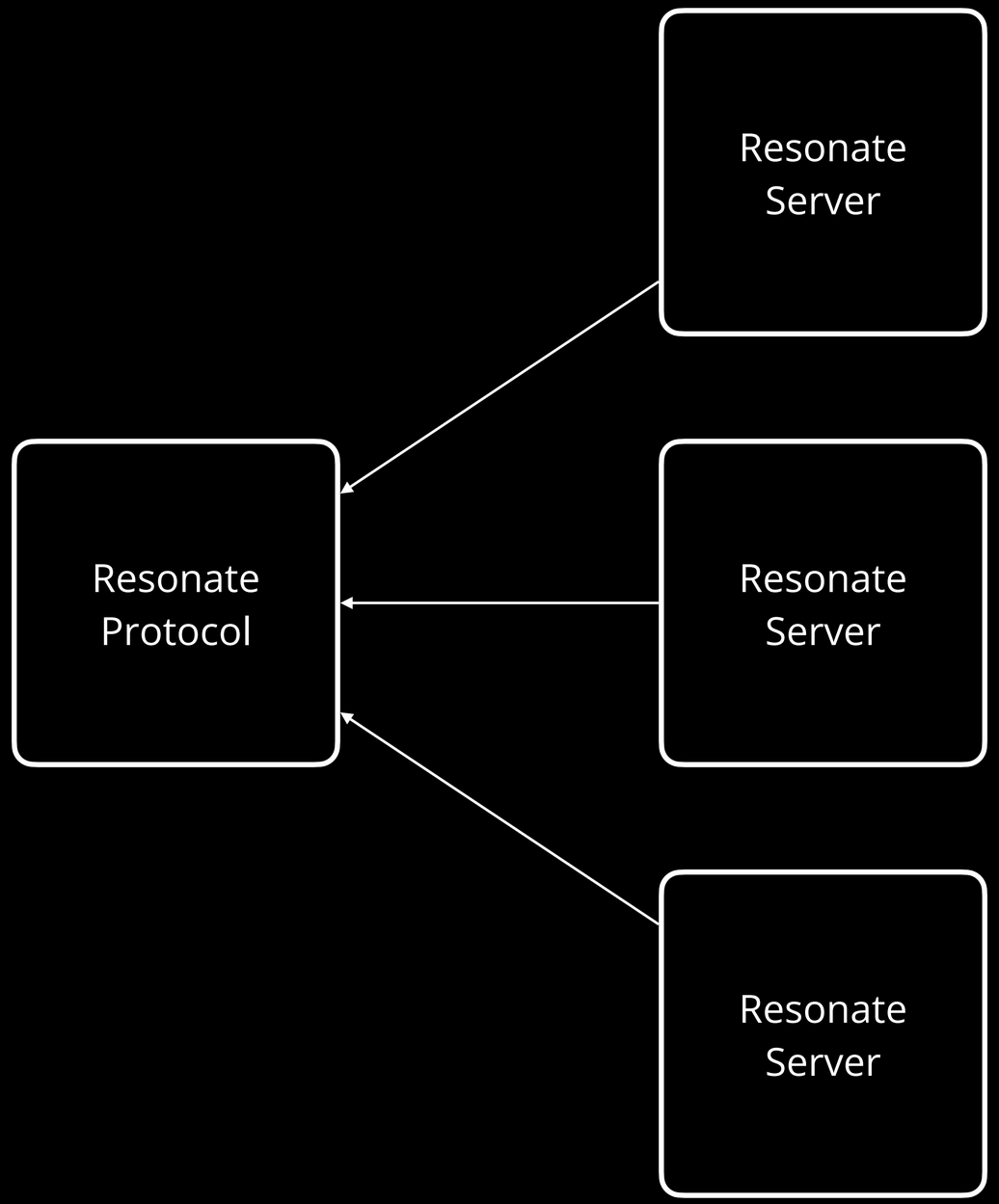
What is the point of libraries now that you can just generate them?

GHUNTLEY.COM


General
Bespoke

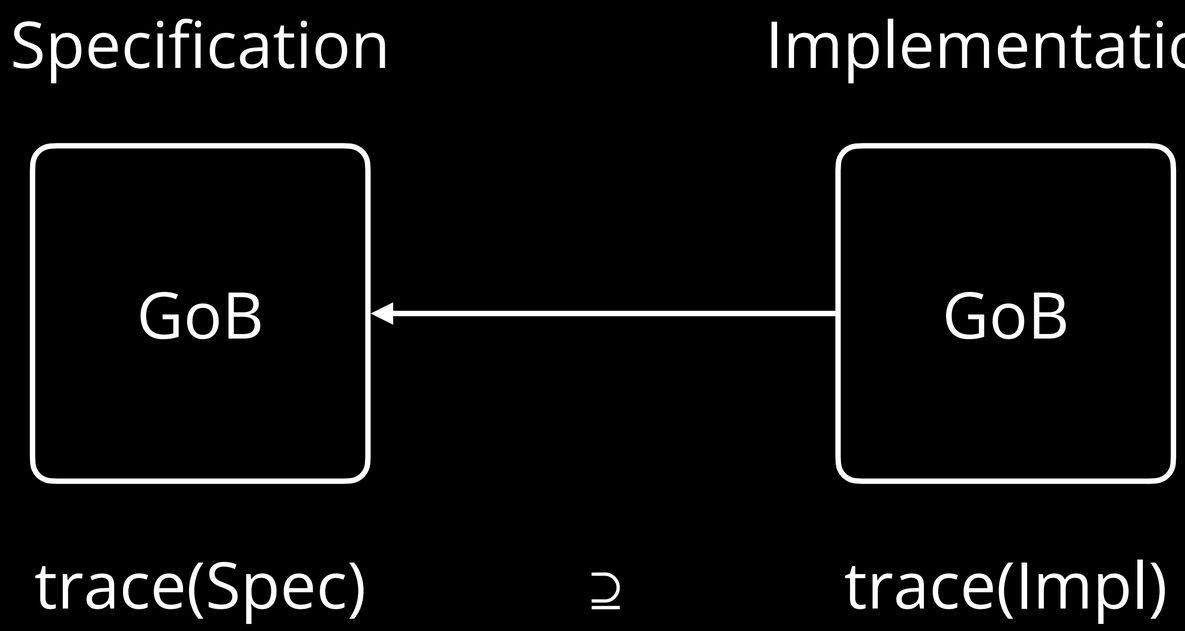

Resonate
Server

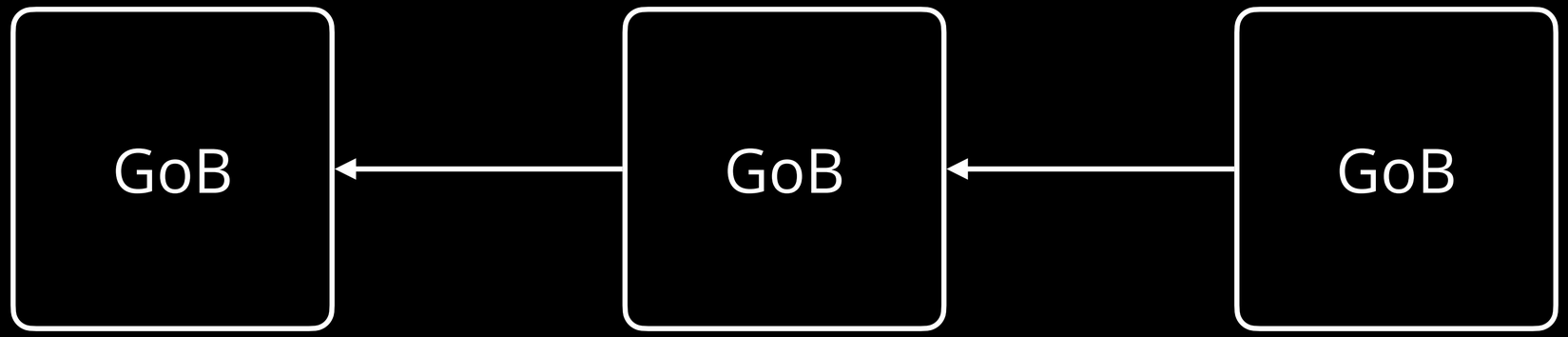

Resonate
Server
Resonate
Server
Resonate
Server

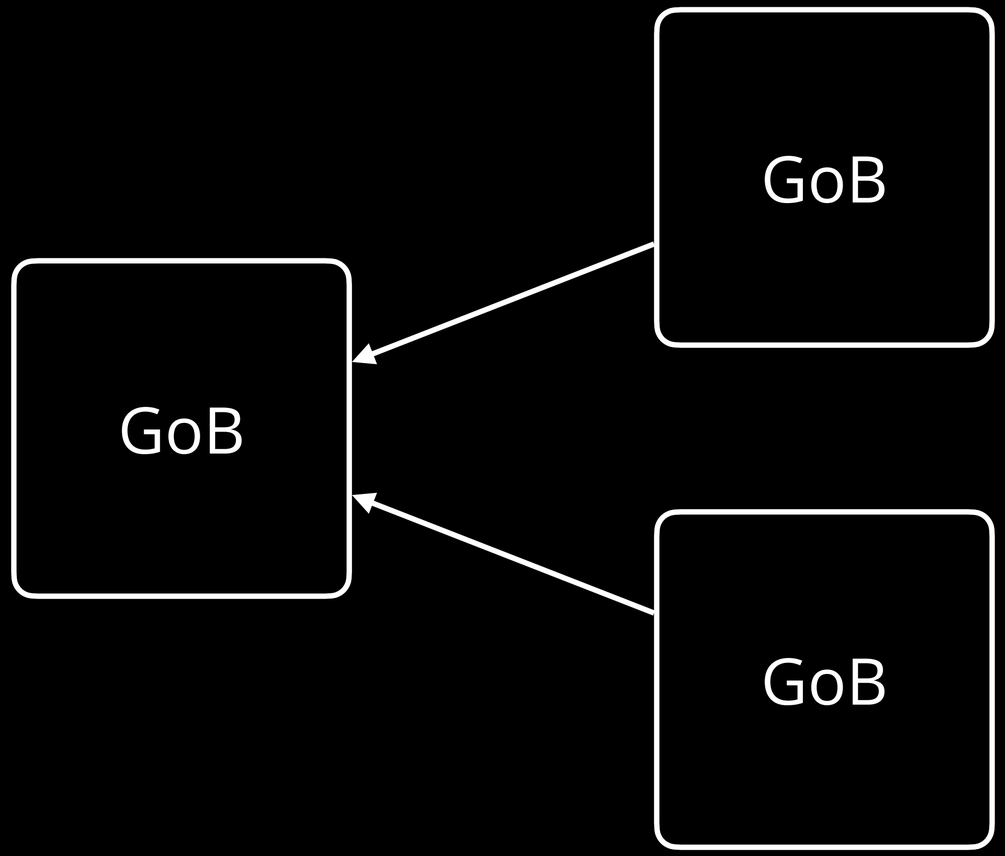

Resonate
Server
Resonate
Protocol
Resonate
Server
Resonate
Server

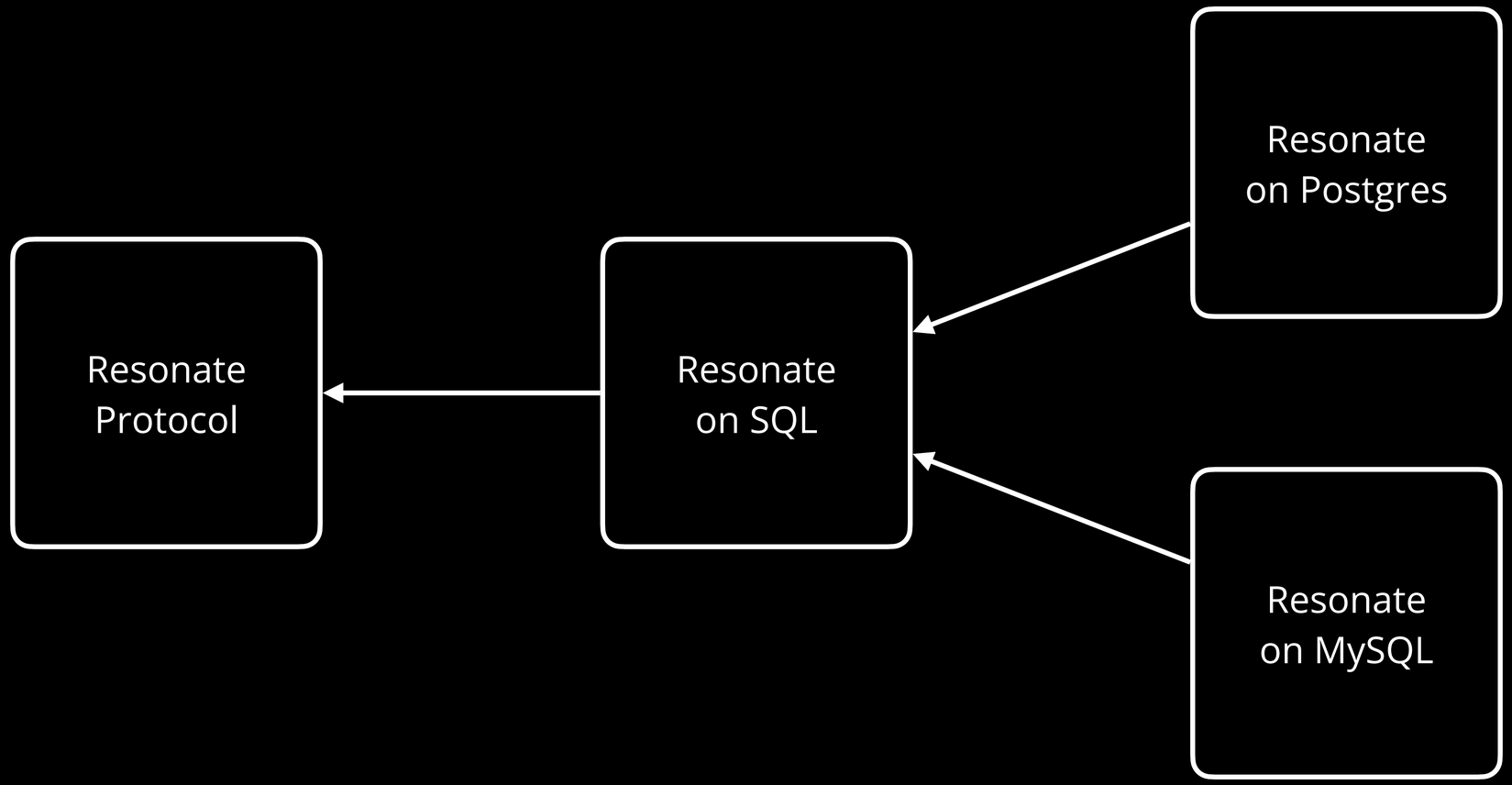
Spec
Agent
Impl

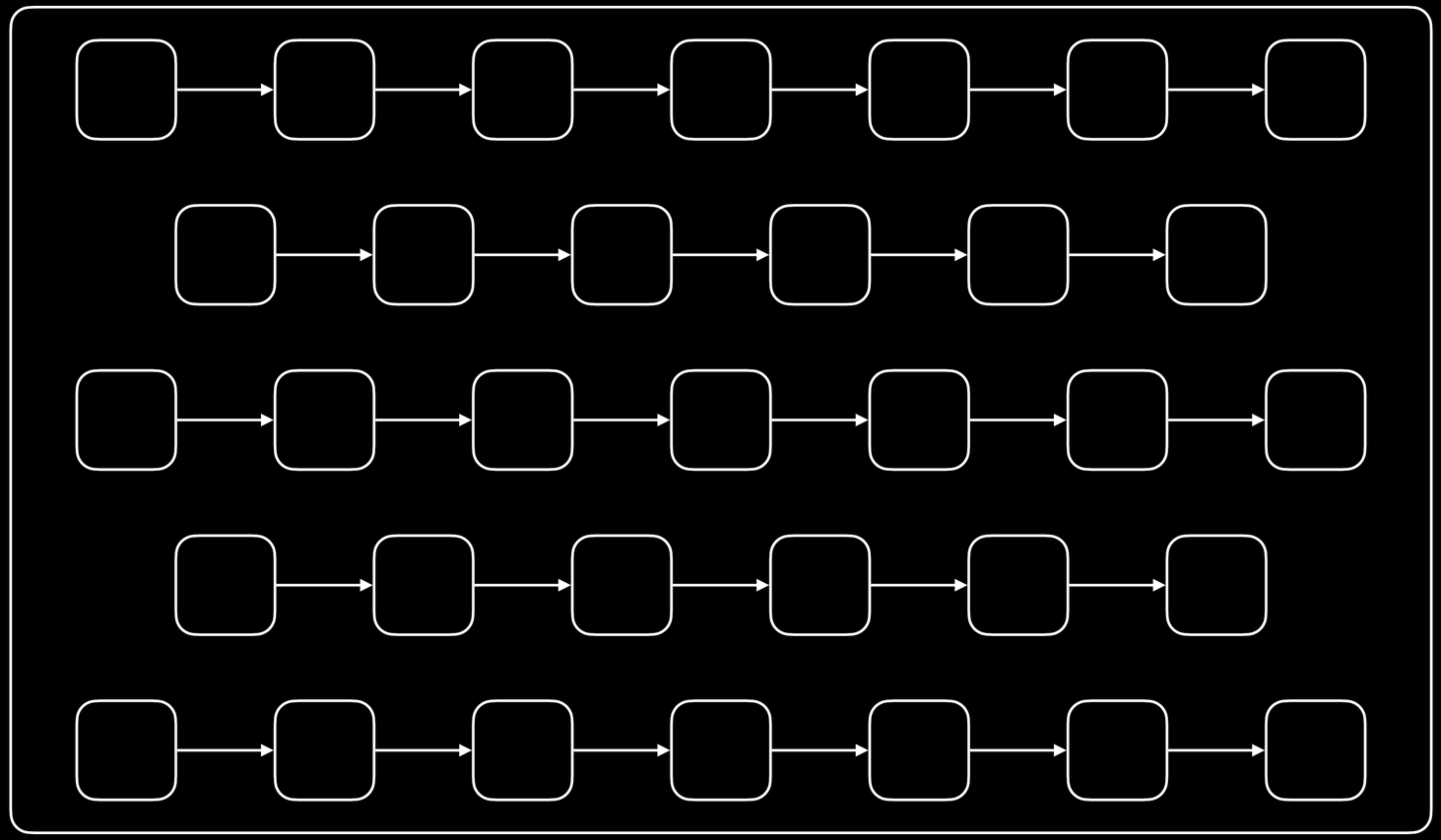
trace(GoB)

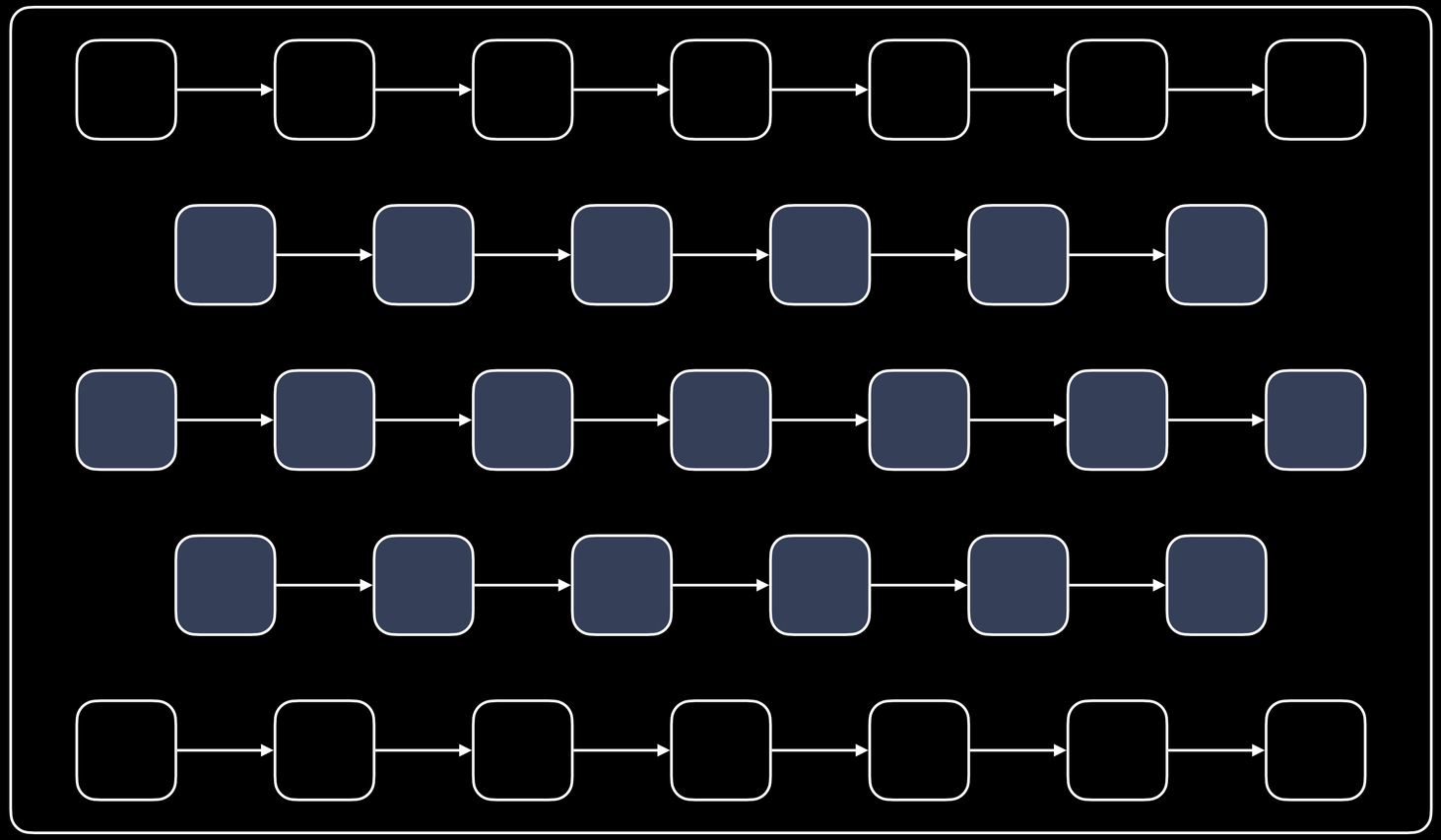

Specification
Implementation
GoB
GoB
trace(Spec)
⊇
trace(Impl)

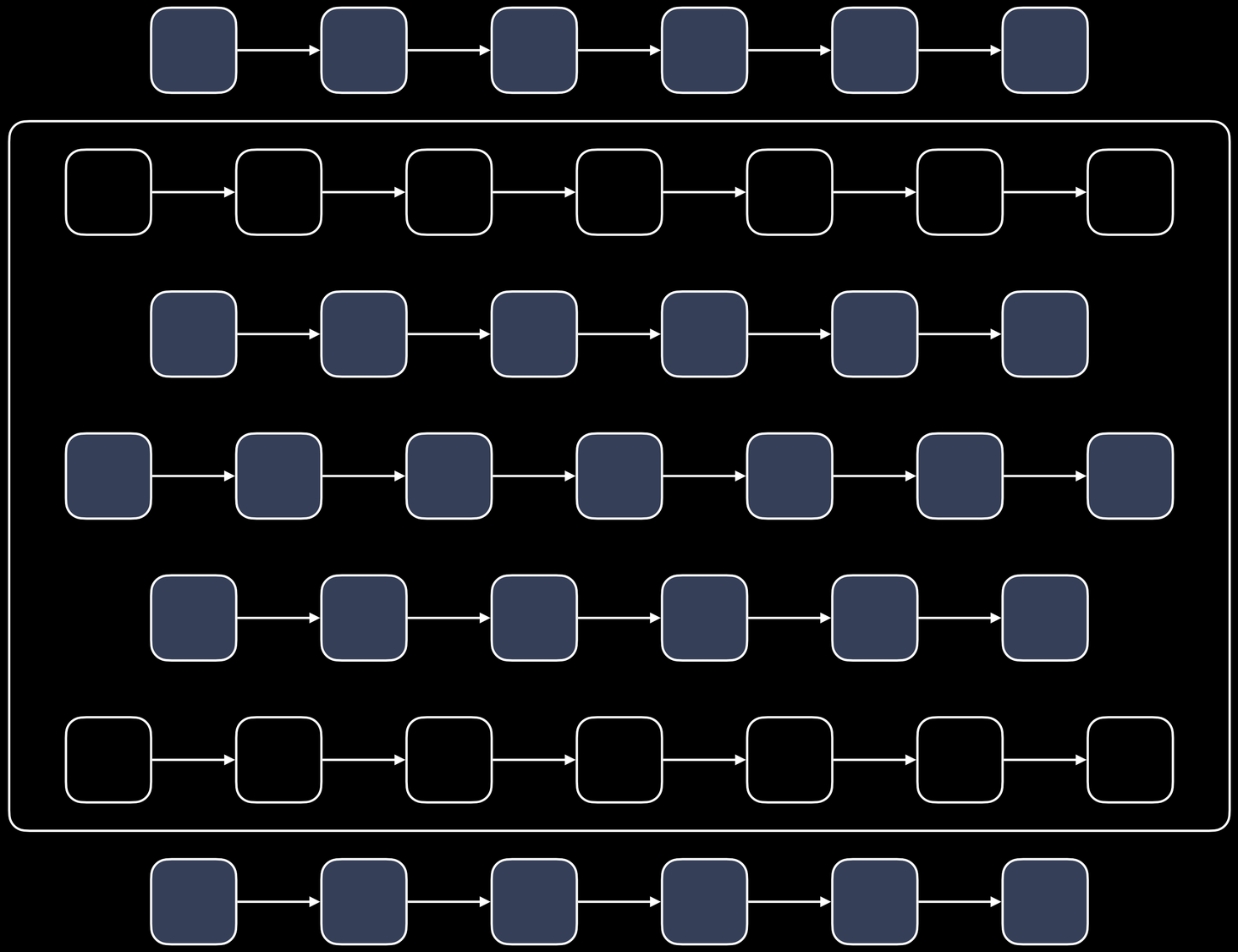

GoB
GoB
GoB


GoB
GoB
GoB

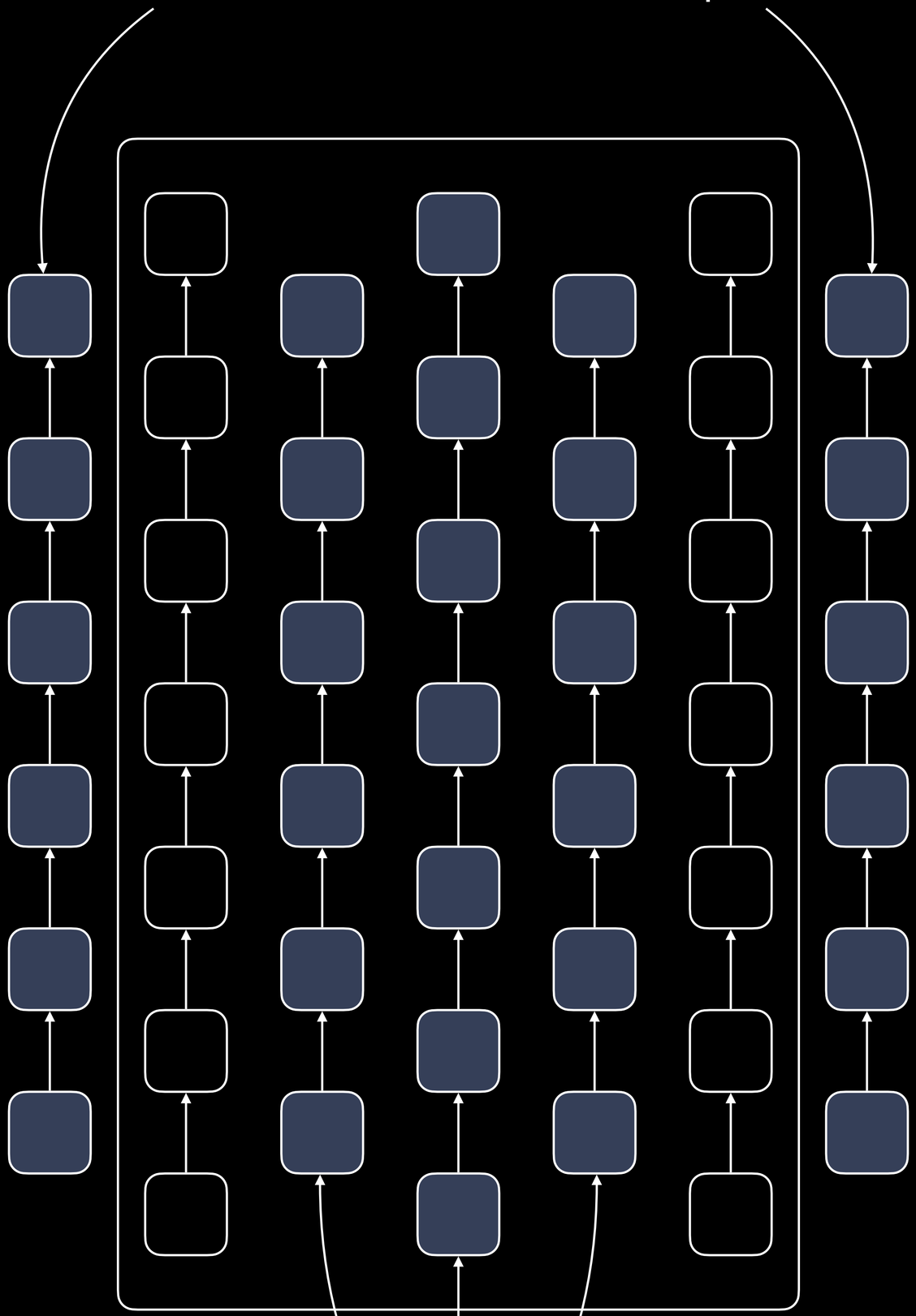

Resonate
Protocol
Resonate
on SQL
Resonate
on Postgres
Resonate
on MySQL







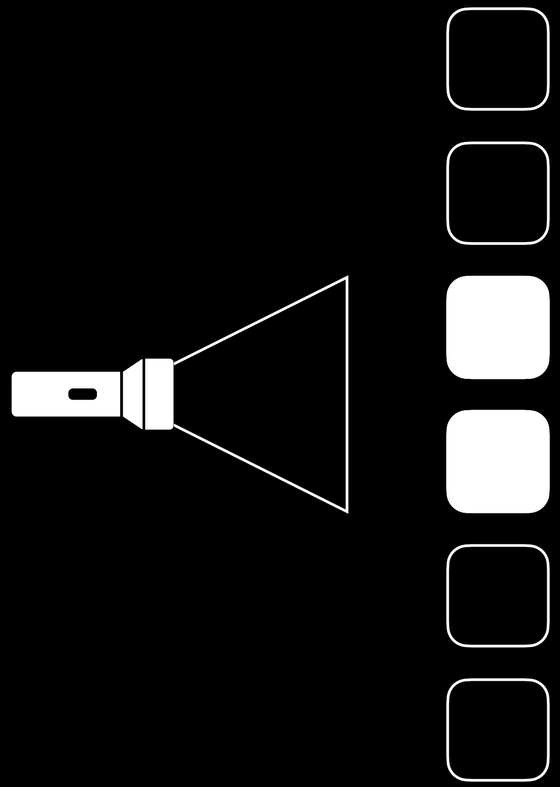
Software Testing
aims to find
at least one traces are outside the specification

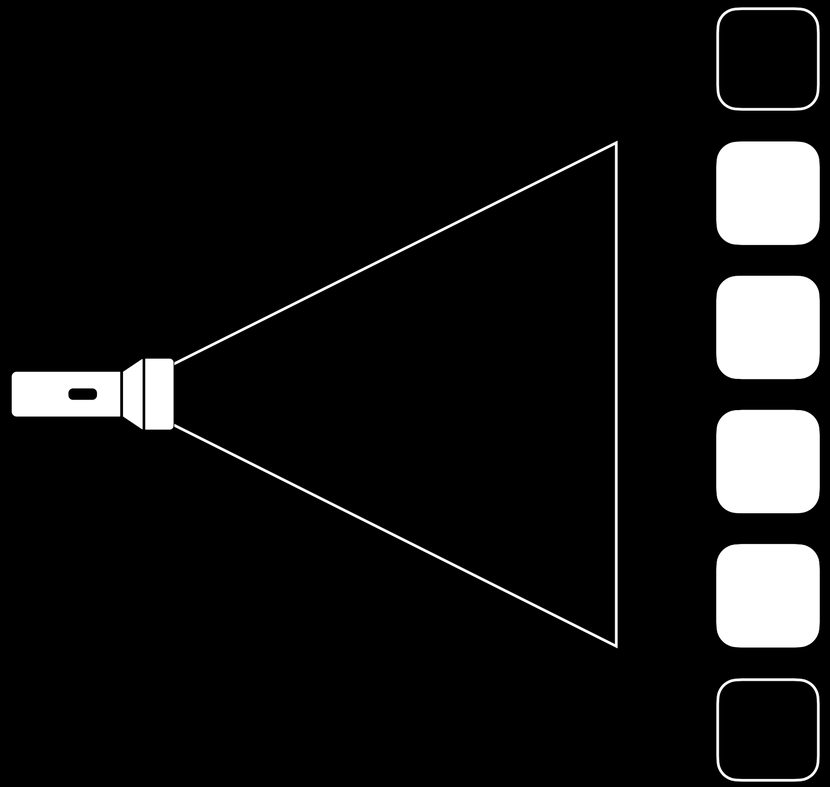


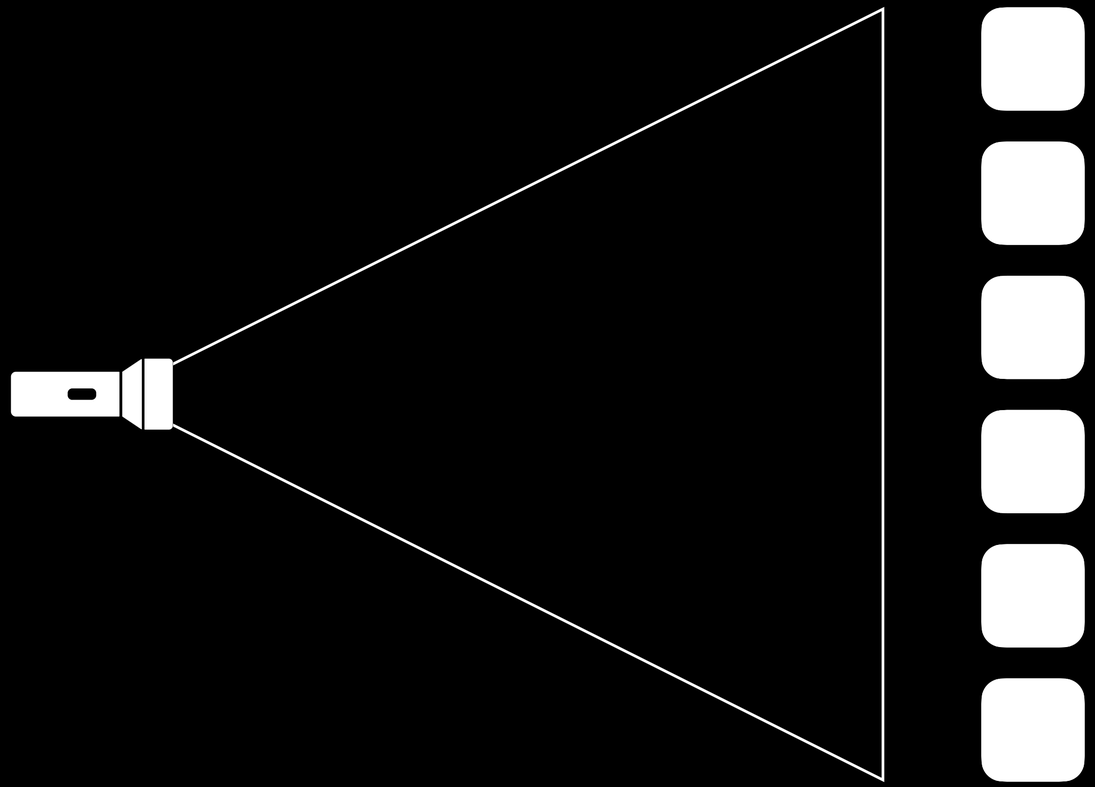
Software Verification
aims to prove
all traces are within the specification

$$I = \{ i \mid i \models S \}$$

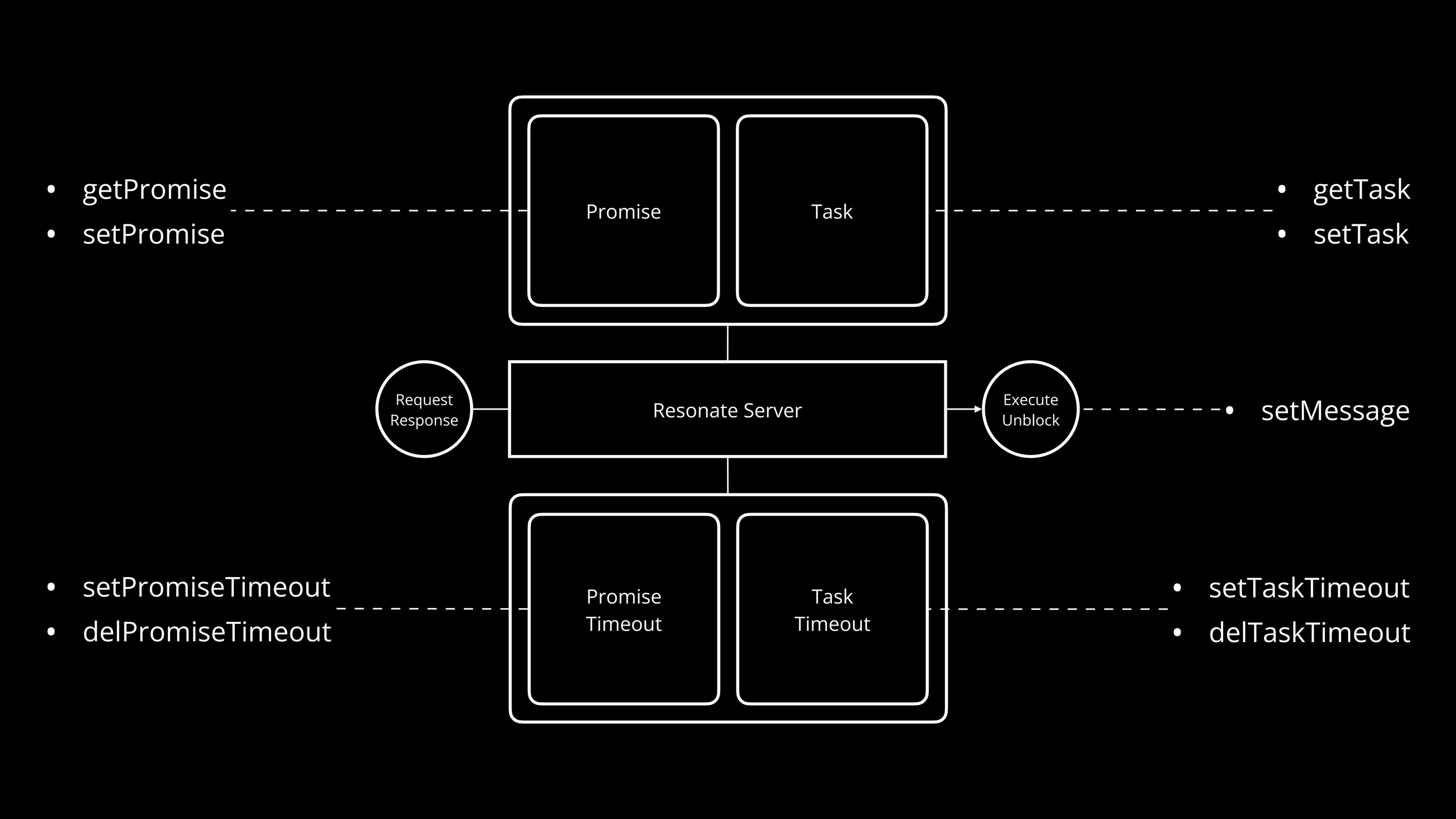
$$I = \{\, i \mid \underbrace{i \models S}_{\text{constraint}} \wedge \underbrace{f(i) \geq \tau}_{\text{objective}} \,\}$$







promise.create(id)
task(execute(id))
Promise
id
task(execute(id))
promise.settle(id)



- getPromise
- setPromise

- getTask
- setTask

- setMessage

- setPromiseTimeout
- delPromiseTimeout

- setTaskTimeout
- delTaskTimeout

Promise

- promise.get
- promise.create
- promise.settle
- promise.register_callback
- promise.register_listener

Task

- task.get
- task.create
- task.acquire
- task.release
- task.suspend
- task.fulfill
- task.halt
- task.continue
- task.fence
- task.heartbeat

```
def promiseCreate (req : PromiseCreateReq) (now : Nat) : M PromiseCreateRes := do
  let retryTimeout := (← get).config.retryTimeout
  match ← getPromise req.id with
  | none =>
      if req.timeoutAt > now then
        let p : PromiseObject :=
          { id := req.id
            state := .pending
            param := req.param
            tags := req.tags
            timeoutAt := req.timeoutAt
            createdAt := now }
        setPromise p
        if p.external then
          setPromiseTimeout p.id p.timeoutAt
        match p.tags.get? "resonate:target" with
        | none =>
            return { status := 200, promise := some p.toRecord }
        | some target =>
            let t : TaskObject := { id := p.id, state := .pending, version := 0 }
            setTask t
            setTaskTimeout t.id 0 (now + retryTimeout)
            setMessage target (.execute t.id t.version)
            return { status := 200, promise := some p.toRecord }
      else
        ...
```

```
def promiseCreate (req : PromiseCreateReq) (now : Nat) : M PromiseCreateRes := do
  let retryTimeout := (← get).config.retryTimeout
  match ← getPromise req.id with
  | none =>
      if req.timeoutAt > now then
        let p : PromiseObject :=
          { id := req.id
            state := .pending
            param := req.param
            tags := req.tags
            timeoutAt := req.timeoutAt
            createdAt := now }
        setPromise p
        if p.external then
          setPromiseTimeout p.id p.timeoutAt
        match p.tags.get? "resonate:target" with
        | none =>
            return { status := 200, promise := some p.toRecord }
        | some target =>
            let t : TaskObject := { id := p.id, state := .pending, version := 0 }
            setTask t
            setTaskTimeout t.id 0 (now + retryTimeout)
            setMessage target (.execute t.id t.version)
            return { status := 200, promise := some p.toRecord }
      else
        ...
```

```
def promiseSettle (req : PromiseSettleReq) (now : Nat) : M PromiseSettleRes := do
  if !req.state.settable then
    return { status := 400 }
  match ← getPromise req.id with
  | none =>
      return { status := 404 }
  | some p =>
      if p.state == .pending ∧ p.timeoutAt > now then
        let listeners := p.listeners
        let callbacks := p.callbacks
        let p := { p with state := req.state, value := req.value, settledAt := some now, ... }
        setPromise p
        delPromiseTimeout p.id
        match ← getTask p.id with
        | some t =>
            setTask { t with state := .fulfilled, pid := none, ttl := none, resumes := [] }
            delTaskTimeout t.id
        | none =>
            pure ()
        for address in listeners do
          setMessage address (.unblock p.toRecord)
        for awaiterId in callbacks do
          defer { awaited := p.id, awaiter := awaiterId }
        return { status := 200, promise := some p.toRecord }
      else
        return { status := 200, promise := some (p.project now).toRecord }
```

```
def promiseSettle (req : PromiseSettleReq) (now : Nat) : M PromiseSettleRes := do
  if !req.state.settable then
    return { status := 400 }
  match ← getPromise req.id with
  | none =>
      return { status := 404 }
  | some p =>
      if p.state == .pending ∧ p.timeoutAt > now then
        let listeners := p.listeners
        let callbacks := p.callbacks
        let p := { p with state := req.state, value := req.value, settledAt := some now, ... }
        setPromise p
        delPromiseTimeout p.id
        match ← getTask p.id with
        | some t =>
            setTask { t with state := .fulfilled, pid := none, ttl := none, resumes := [] }
            delTaskTimeout t.id
        | none =>
            pure ()
        for address in listeners do
          setMessage address (.unblock p.toRecord)
        for awaiterId in callbacks do
          defer { awaited := p.id, awaiter := awaiterId }
        return { status := 200, promise := some p.toRecord }
      else
        return { status := 200, promise := some (p.project now).toRecord }
```

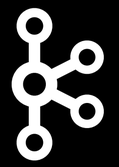
```
structure StateAction where
  state : ServerState
  req   : Request
  res   : Response
  now   : Nat

abbrev Trace := Nat → StateAction

def Valid (tr : Trace) : Prop :=
  ∀ t : Nat,
    let (res, next) := handle (tr t).now (tr t).req (tr t).state
      (tr t).now ≤ (tr (t + 1)).now ∧ (tr t).res = res ∧ (tr (t + 1)).state = next
```

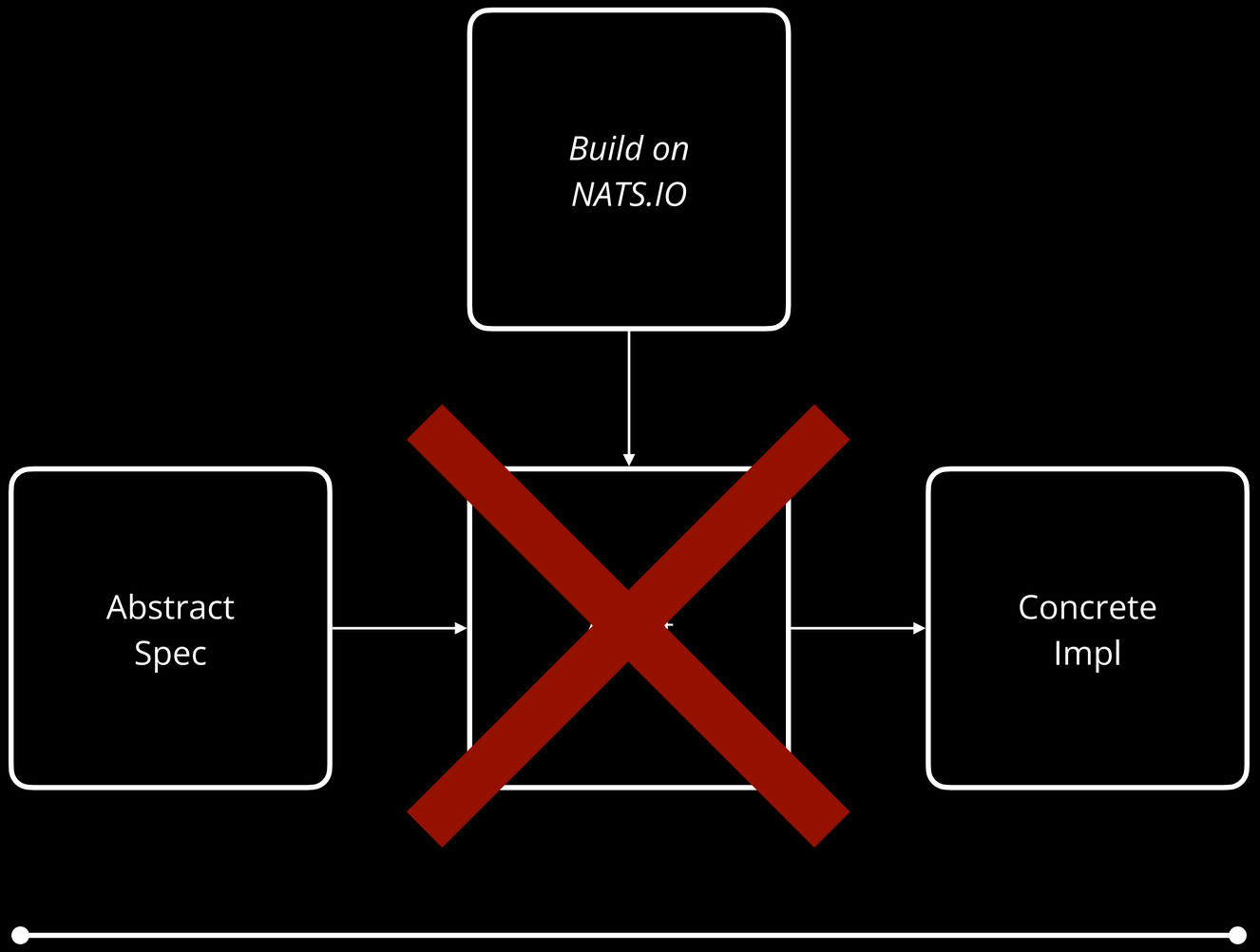
```
def Stable (tr : Trace) : Prop :=
  ∀ i j : Nat, ∀ id : String, ∀ st₁ st₂ : PromiseState,
    i ≤ j →
      Externalized (tr i).res id st₁ →
        Settled st₁ →
          Externalized (tr j).res id st₂ →
            st₂ = st₁
```

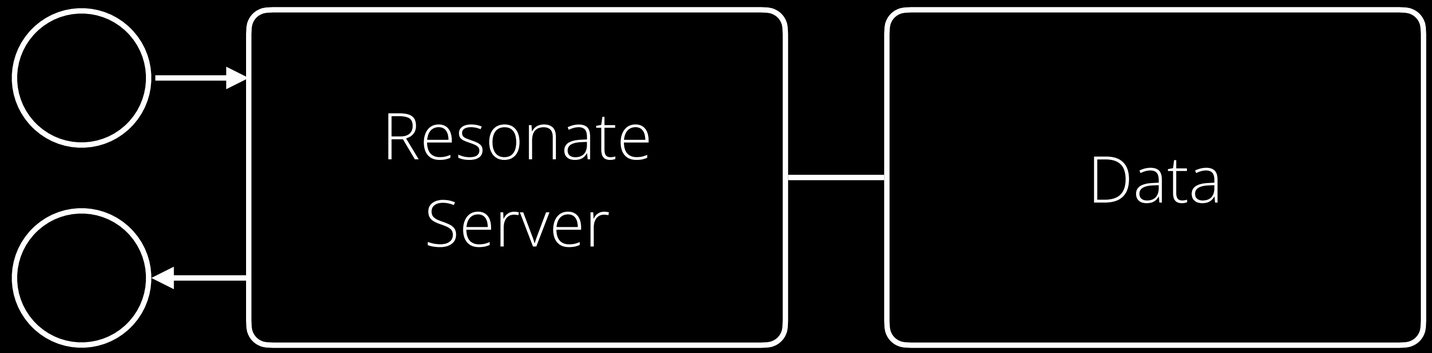
```
theorem stable : ∀ tr : Trace, Valid tr → Stable tr :=
  Fable (2,738 loc)
```

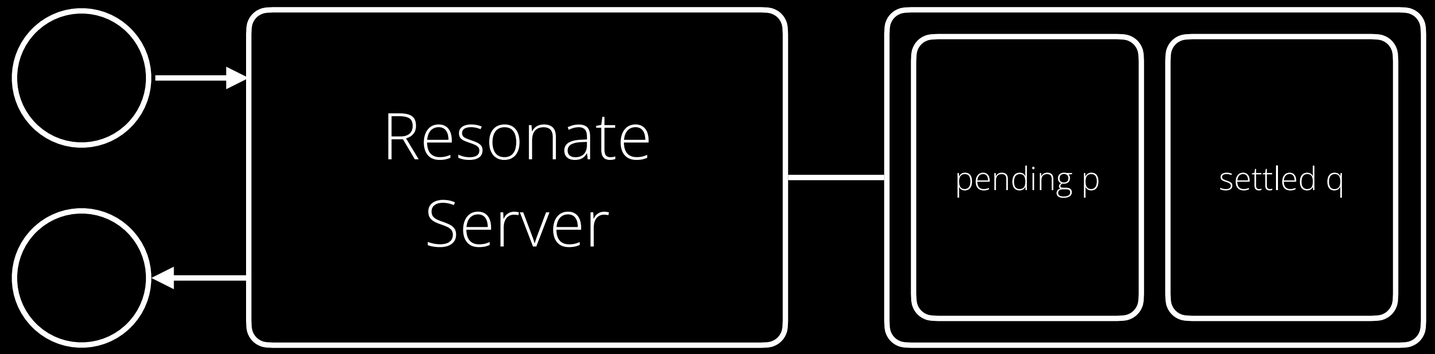
Durable Execution

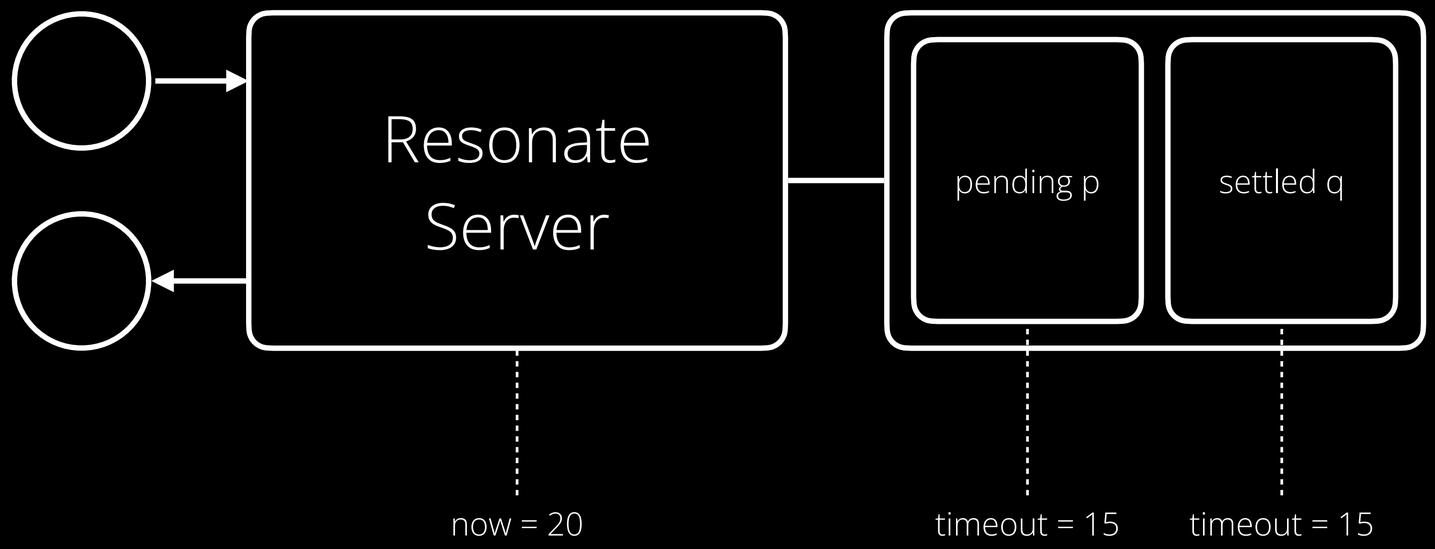
Powered by Resonate

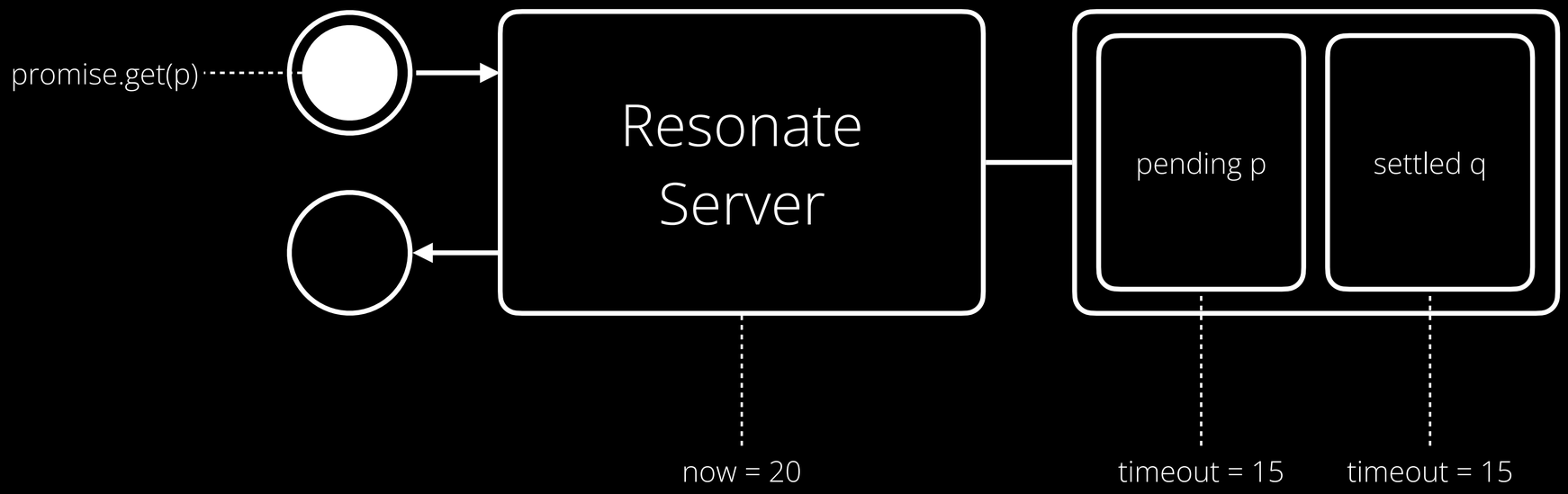


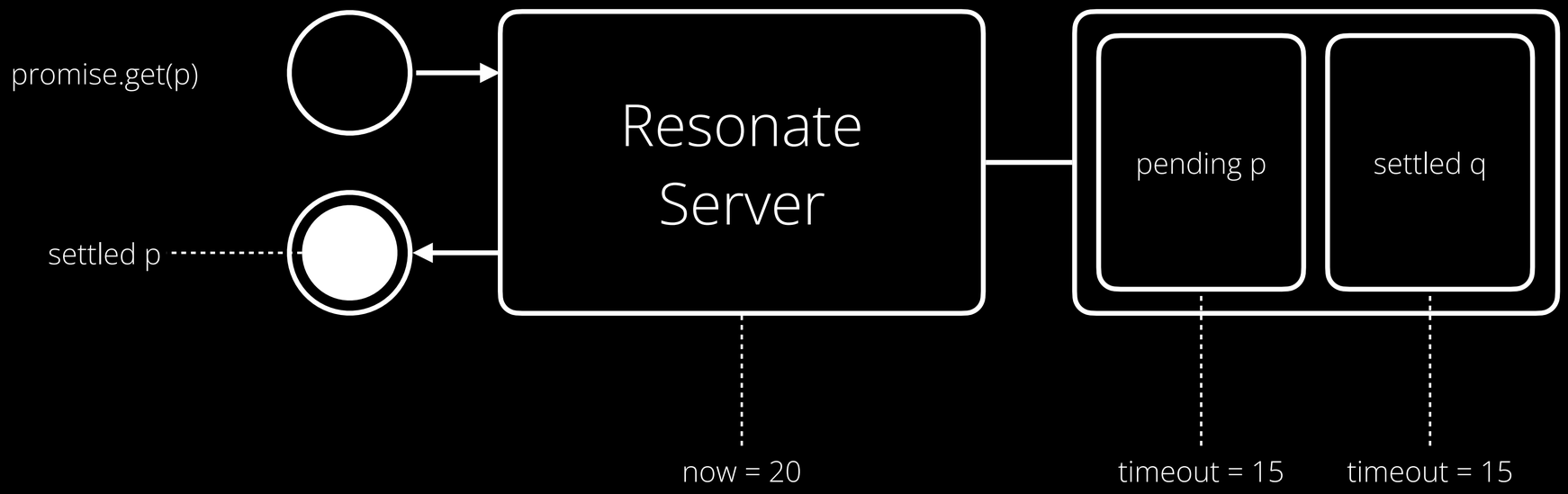

Build on
NATS.IO
Abstract
Spec
Concrete
Impl

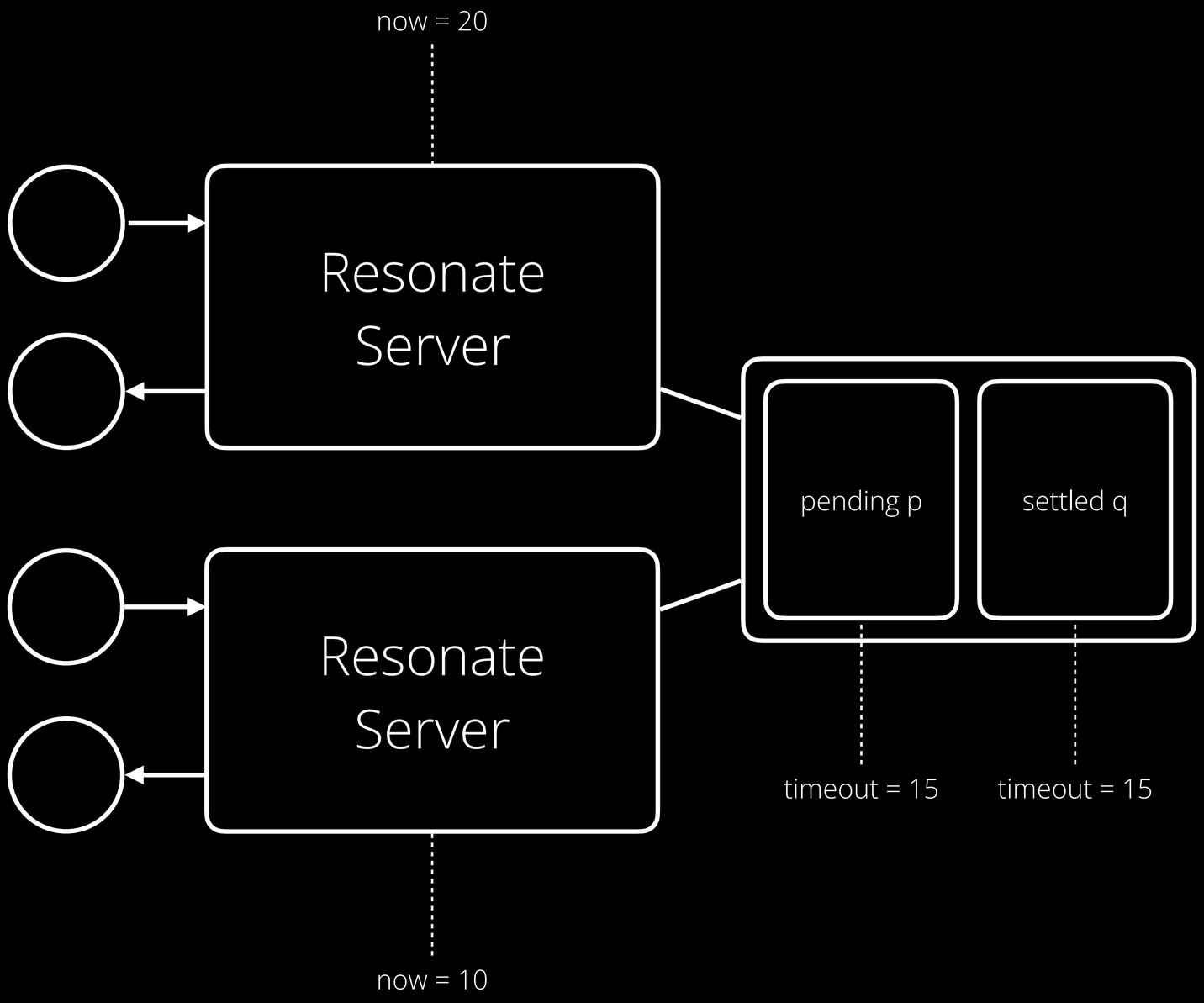

Resonate
Server
Data

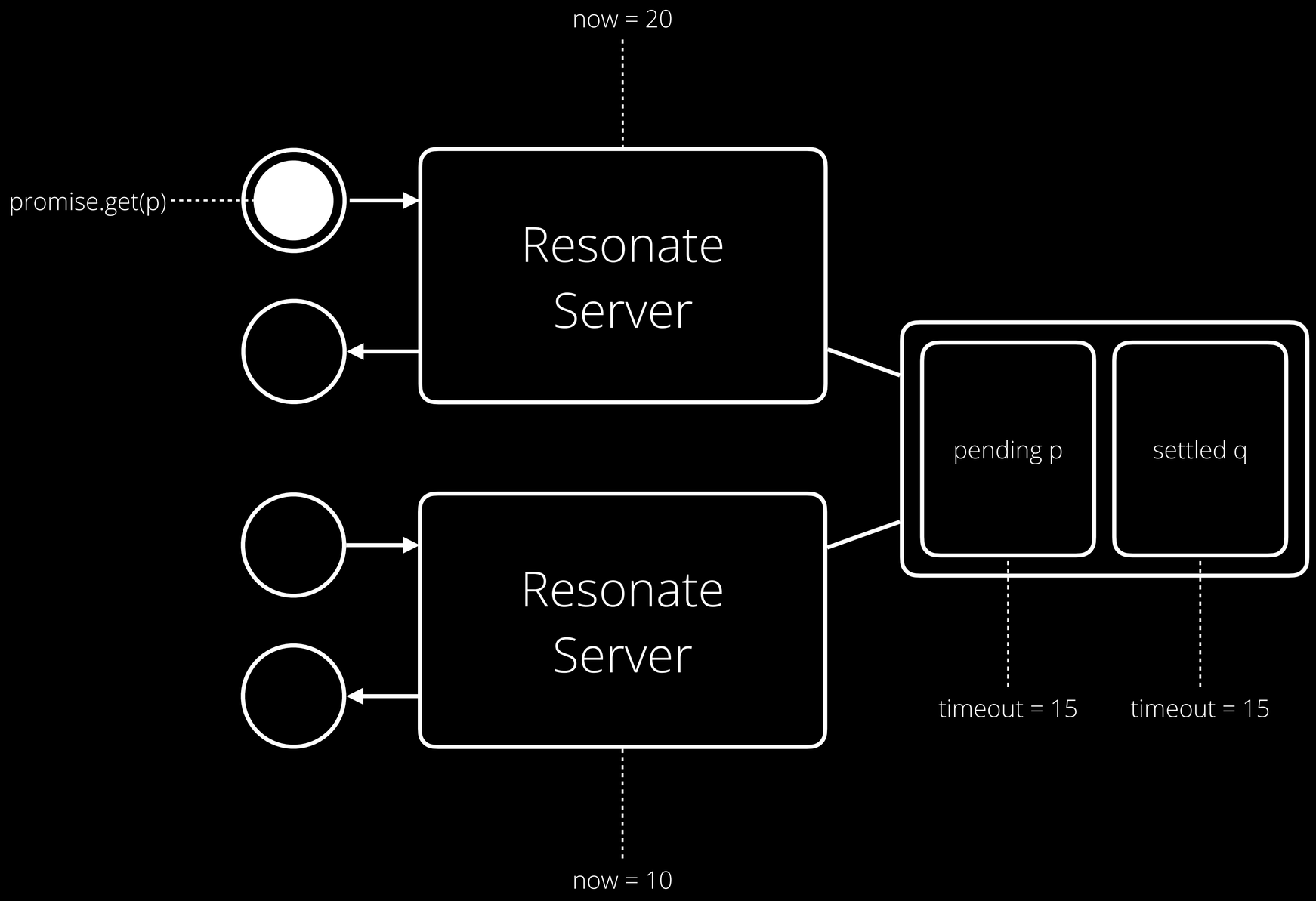

Resonate
Server
pending p
settled q

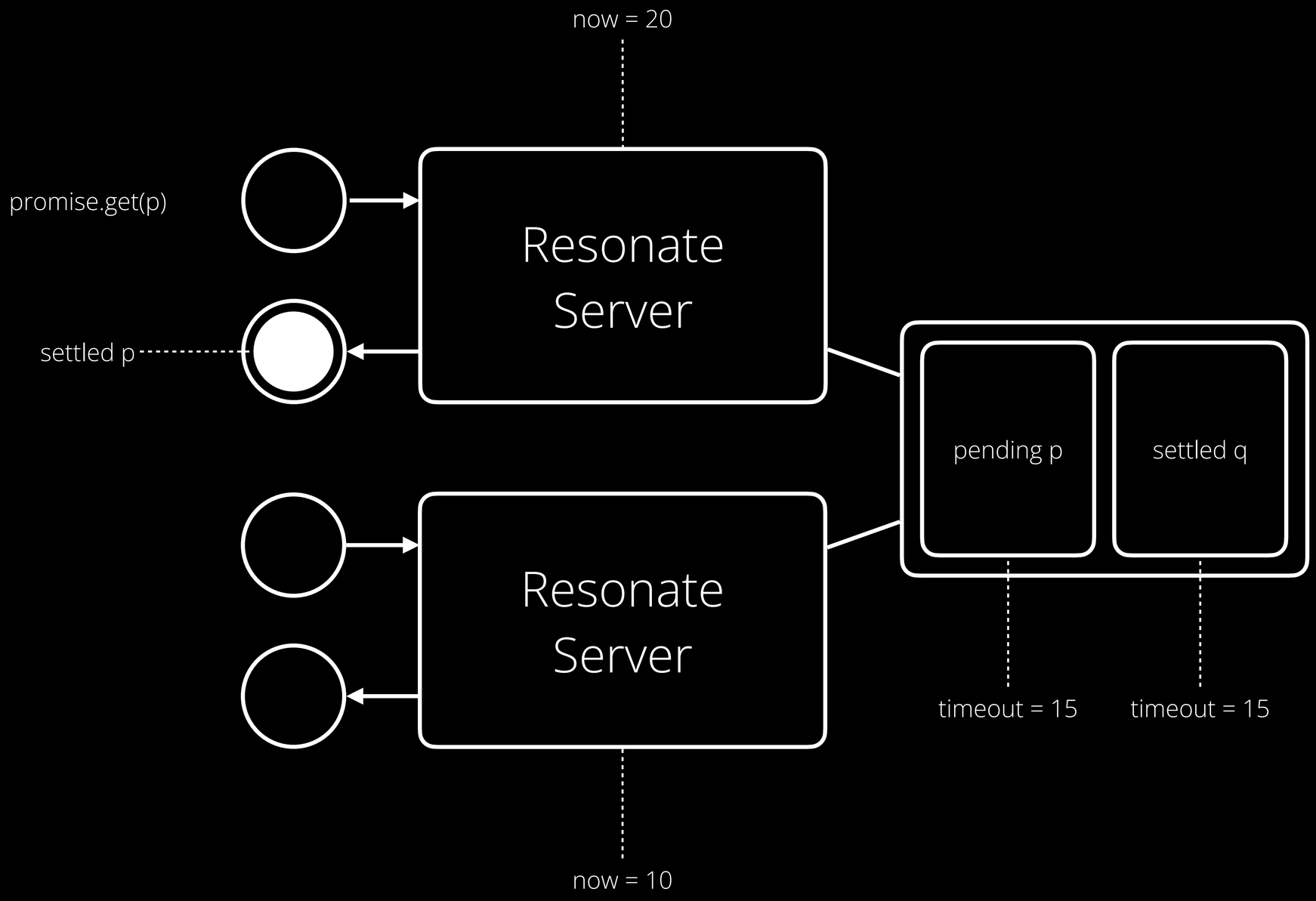

Resonate
Server
pending p
settled q
now = 20
timeout = 15
timeout = 15

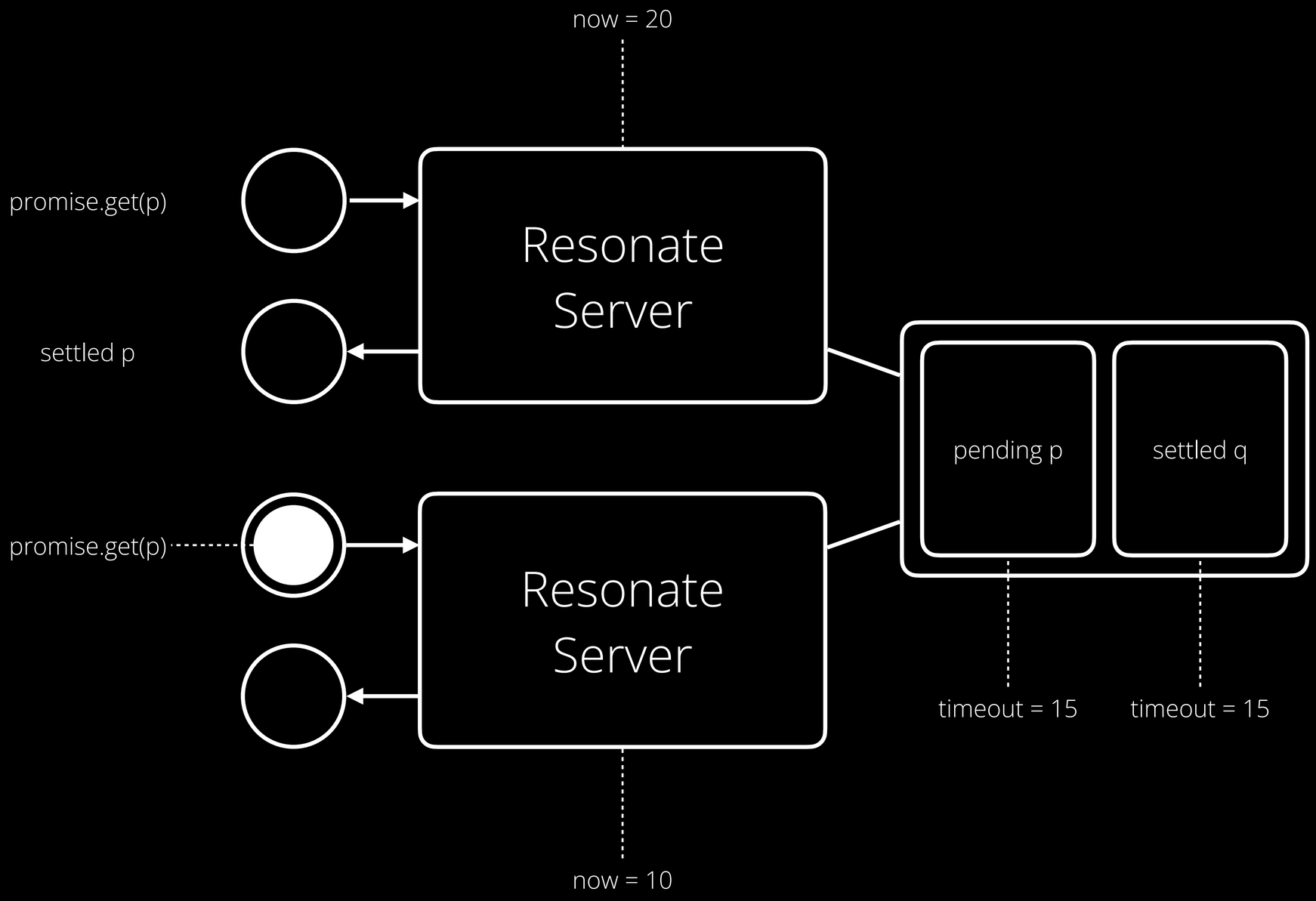

promise.get(p)
Resonate Server
pending p
settled q
now = 20
timeout = 15
timeout = 15

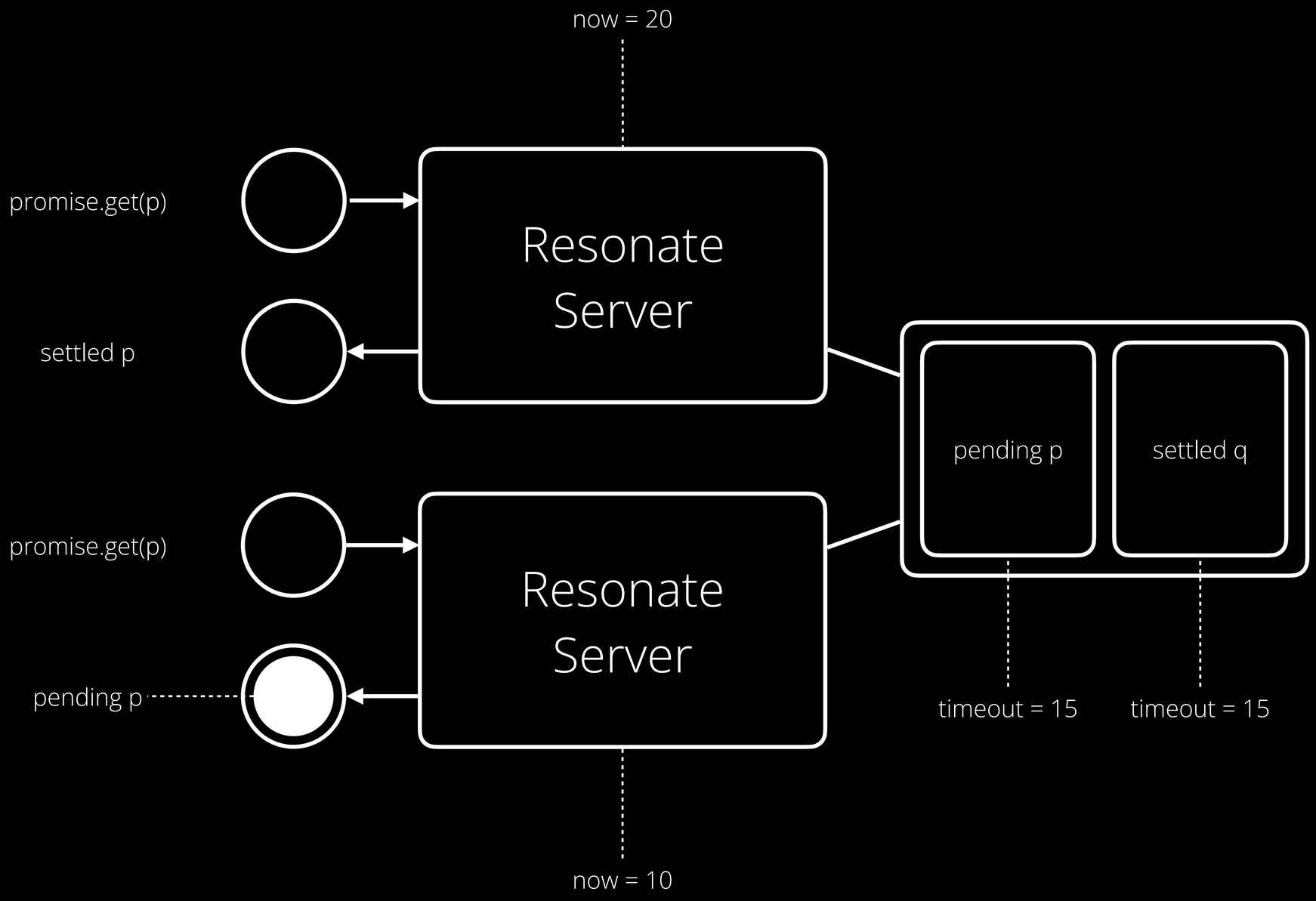

promise.get(p)
settled p
Resonate Server
pending p
settled q
now = 20
timeout = 15
timeout = 15

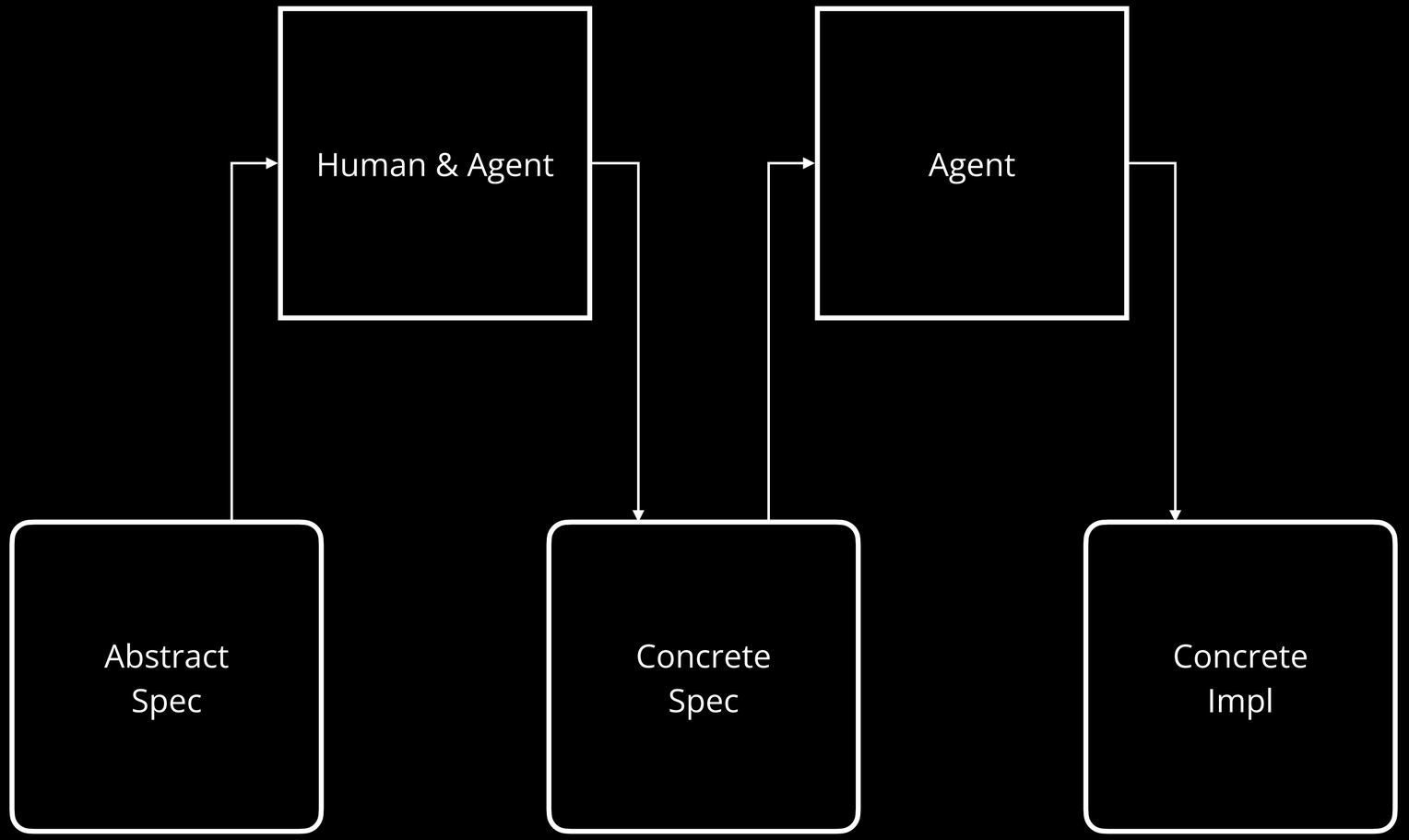

now = 20
Resonate Server
Resonate Server
pending p
settled q
timeout = 15
timeout = 15
now = 10

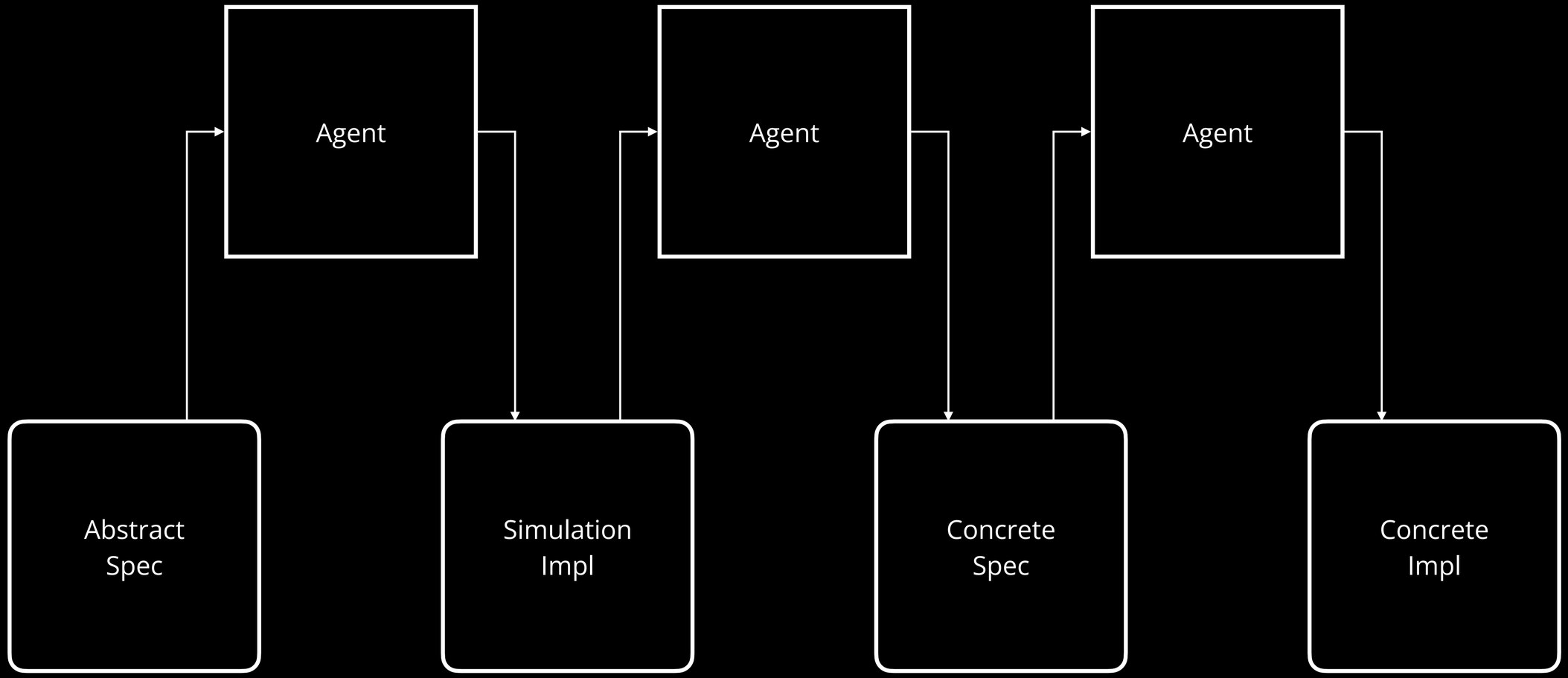

now = 20
promise.get(p)
Resonate Server
Resonate Server
pending p
settled q
timeout = 15
timeout = 15
now = 10

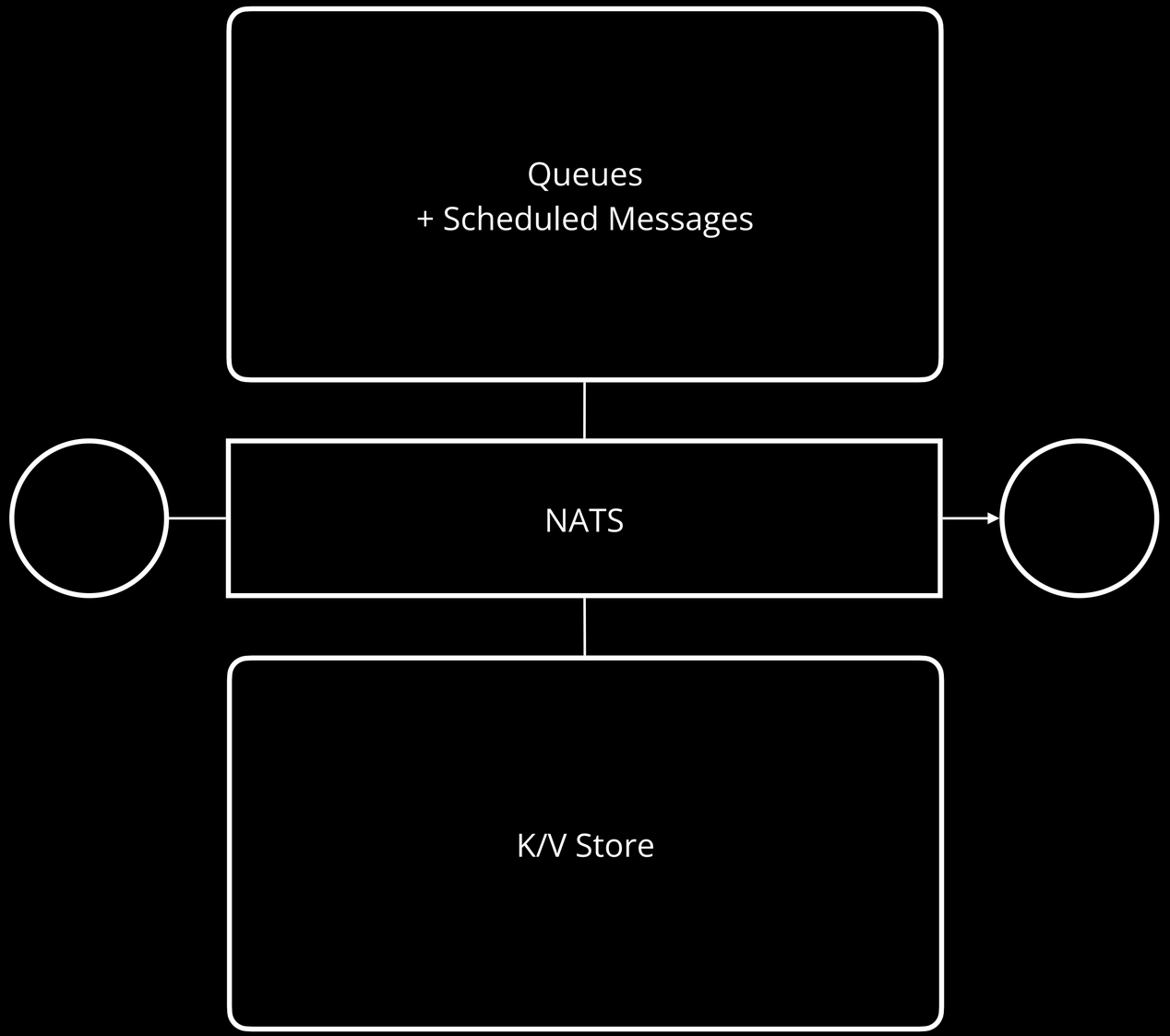

now = 20
promise.get(p)
Resonate Server
settled p
pending p
settled q
Resonate Server
timeout = 15
timeout = 15
now = 10


now = 20
promise.get(p)
Resonate Server
settled p
promise.get(p)
Resonate Server
pending p
settled q
timeout = 15
timeout = 15
now = 10


now = 20
promise.get(p)
Resonate Server
settled p
promise.get(p)
Resonate Server
pending p
pending p
settled q
timeout = 15
timeout = 15
now = 10


Human & Agent
Agent
Abstract
Spec
Concrete
Spec
Concrete
Impl


Agent
Agent
Agent
Abstract
Spec
Simulation
Impl
Concrete
Spec
Concrete
Impl


Queues
+ Scheduled Messages
NATS
K/V Store

```
>>> kvs = KVStore(...)
>>> t = kvs.create(“key”, ”foo”)
>>> t = kvs.update(“key”, “bar”, t)
>>> t = kvs.update(“key”, “baz”, t)
>>> kvs.get(“key”)
(“baz”, 2) # FRESH
```

```
>>> kvs = KVStore(...)
>>> t = kvs.create(“key”, ”foo”)
>>> t = kvs.update(“key”, “bar”, t)
>>> t = kvs.update(“key”, “baz”, t)
>>> kvs.get(“key”)
(“foo”, 0) # STALE
```

```
>>> kvs = KVStore(...)
>>> t = kvs.create(“key”, ”foo”)
>>> t = kvs.update(“key”, “bar”, t)
>>> t = kvs.update(“key”, “baz”, t)
>>> kvs.get(“key”)
(“baz”, 2) # FRESH
```

```
>>> kvs = KVStore(...)
>>> t = kvs.create(“key”, ”foo”)
>>> t = kvs.update(“key”, “bar”, t)
>>> t = kvs.update(“key”, “baz”, t)
>>> kvs.get(“key”)
(“foo”, 0) # STALE
```

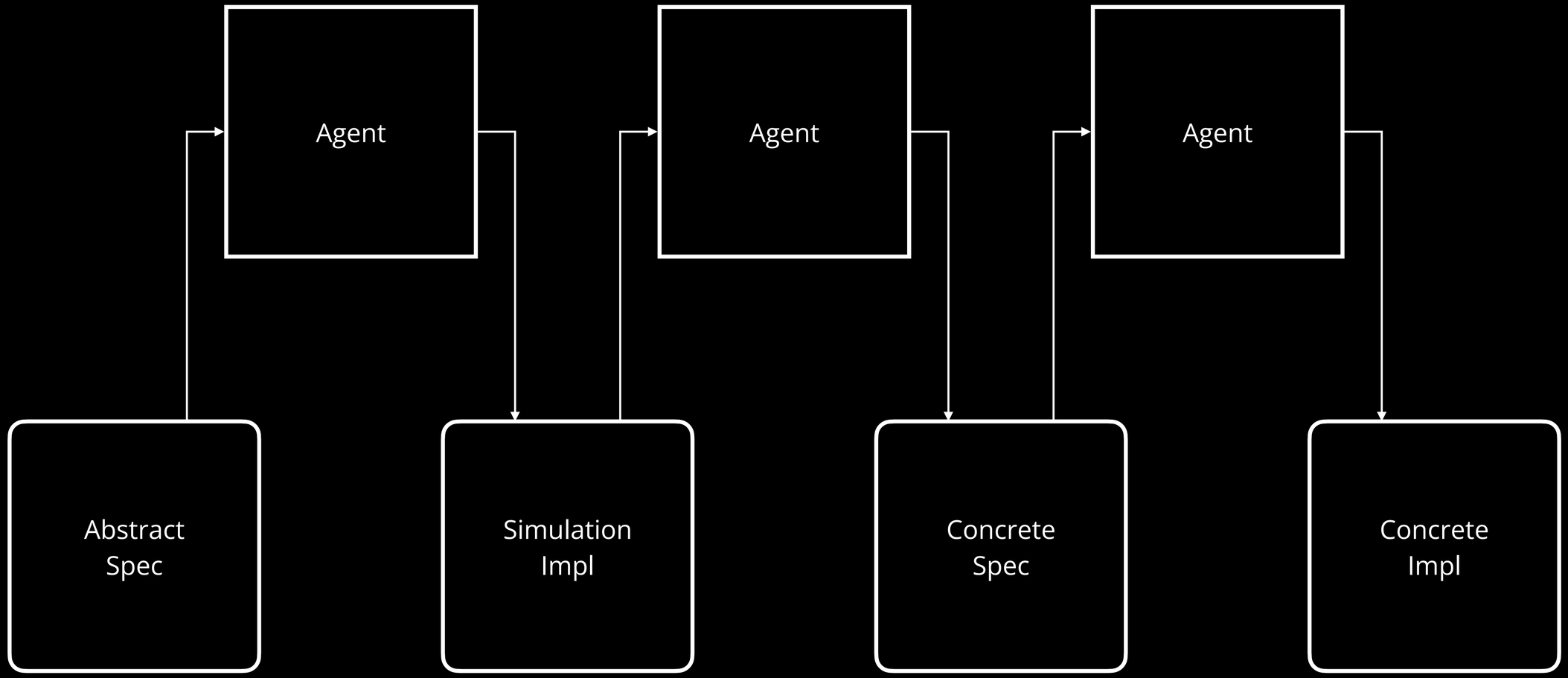
```
>>> kvs.update(“key”, “...”, 0)
KeyError(“key”)
```

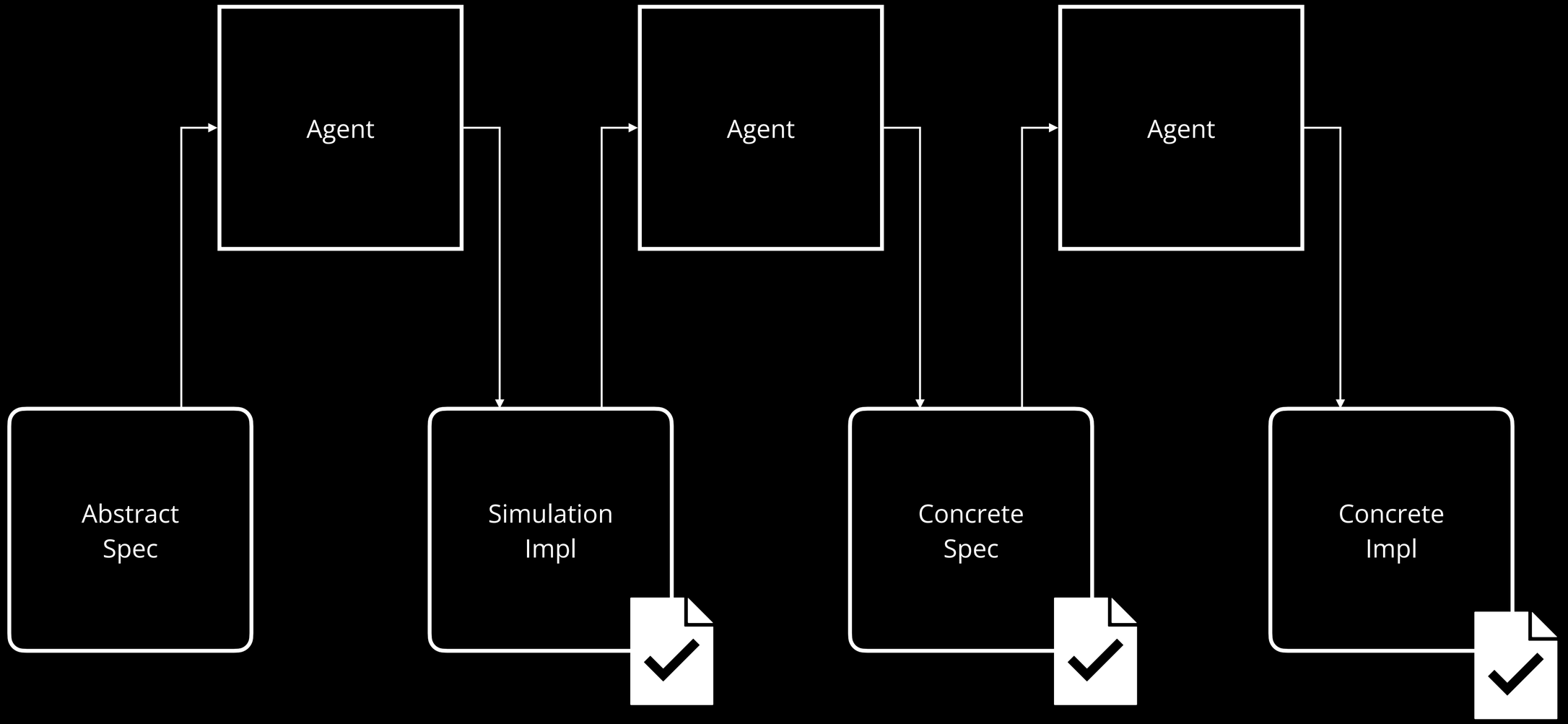
```
class KVStore:
    def __init__(self, ctx, rng, pro):
        self.ctx, self.rng, self.pro, self.store = ctx, rng, pro, {}

    def get(self, key):
        if self.rng.random() < self.pro:
            i = int(self.rng.random() * len(self.store[key]))
        else:
            i = len(self.store[key]) - 1
       return self.store[key][i], i

    def create(self, key, value):
        if key in self.store:
            raise KeyError(key)
        self.store[key] = [value]
        return 0

    def update(self, key, value, index):
        if index != len(self.store[key]) - 1:
            raise KeyError(key)
        self.store[key].append(value)
        return len(self.store[key]) - 1
```

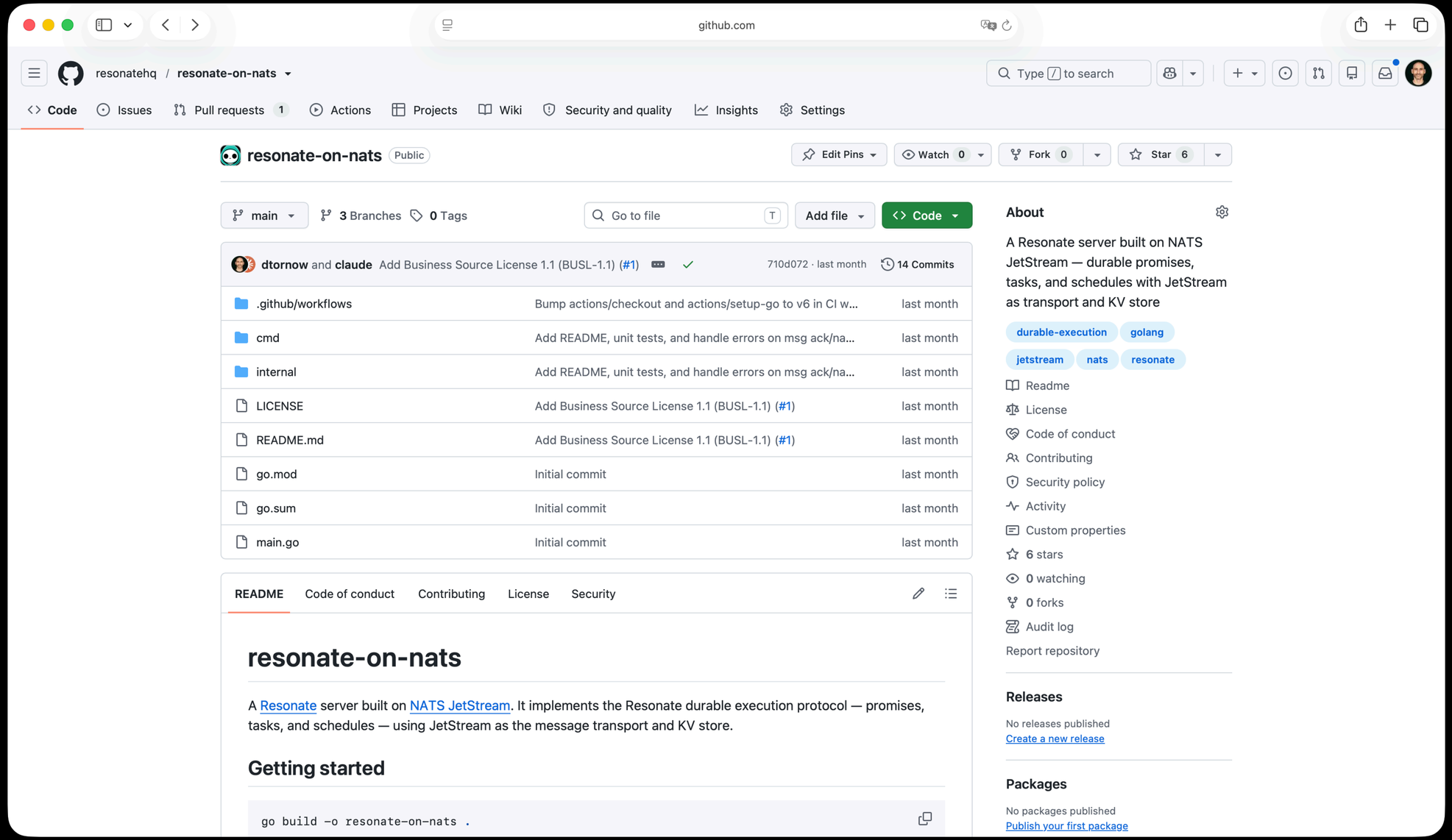
```
class KVStore:

  def get(self, key):
    if self.rng.random() < self.pro:
      i = int(self.rng.random() * len(self.store[key]))
      self.ctx.trace(Get(type=STALE, key=key, result=self.store[key][I], latest=self.store[key][-1]))
    else:
      i = len(self.store[key]) - 1
      self.ctx.trace(Get(type=FRESH, key=key, result=self.store[key][i]))
    return self.store[key][i], i
```

```
class KVStore:

  def get(self, key):
    if self.rng.random() < self.pro:
      i = int(self.rng.random() * len(self.store[key]))
      self.ctx.trace(Get(type=STALE, key=key, result=self.store[key][i], latest=self.store[key][-1]))
    else:
      i = len(self.store[key]) - 1
      self.ctx.trace(Get(type=FRESH, key=key, result=self.store[key][I]))
    return self.store[key][i], i
```

```
{
  "event": “KVStoreGet",
  “key”: “foo-1",
  "type": "STALE",
  "result": {
    "id": "foo-1",
    "status": "pending",
  },
  "latest": {
    "id": "foo-1",
    "status": "settled",
  }
}
```


Agent
Agent
Agent
Abstract
Spec
Simulation
Impl
Concrete
Spec
Concrete
Impl


Agent
Agent
Agent
Abstract
Spec
Simulation
Impl
Concrete
Spec
Concrete
Impl


github.com
resonatehq / resonate-on-nats
Type / to search
Code
Issues
Pull requests 1
Actions
Projects
Wiki
Security and quality
Insights
Settings
resonate-on-nats
Public
Edit Pins
Watch 0
Fork 0
Star 6
main
3 Branches
0 Tags
Go to file
T
Add file
Code
dtornow and claude Add Business Source License 1.1 (BUSL-1.1) (#1)
710d072 · last month
14 Commits
.github/workflows
Bump actions/checkout and actions/setup-go to v6 in CI w...
last month
cmd
Add README, unit tests, and handle errors on msg ack/na...
last month
internal
Add README, unit tests, and handle errors on msg ack/na...
last month
LICENSE
Add Business Source License 1.1 (BUSL-1.1) (#1)
last month
README.md
Add Business Source License 1.1 (BUSL-1.1) (#1)
last month
go.mod
Initial commit
last month
go.sum
Initial commit
last month
main.go
Initial commit
last month
README
Code of conduct
Contributing
License
Security
resonate-on-nats
A Resonate server built on NATS JetStream. It implements the Resonate durable execution protocol — promises, tasks, and schedules — using JetStream as the message transport and KV store.
Getting started
go build -o resonate-on-nats .
About
A Resonate server built on NATS JetStream — durable promises, tasks, and schedules with JetStream as transport and KV store
durable-execution
golang
jetstream
nats
resonate
Readme
License
Code of conduct
Contributing
Security policy
Activity
Custom properties
6 stars
0 watching
0 forks
Audit log
Report repository
Releases
No releases published
Create a new release
Packages
No packages published
Publish your first package

github.com

resonatehq / resonate-pg

Type / to search

Code | Issues 4 | Pull requests | Actions | Projects | Wiki | Security and quality | Insights | Settings

resonate-pg

Public

Edit Pins | Watch 1 | Fork 0 | Star 9

main | 3 Branches | 0 Tags | Go to file T | Add file | Code

dtornow and **claude** A research agent using the Anthropic SDK — 54fe651 · 4 days ago — 6 Commits

assets	initial commit	2 weeks ago
example	A research agent using the Anthropic SDK	4 days ago
test	Improve protocol conformance	last week
.gitignore	initial commit	2 weeks ago
LICENSE	initial commit	2 weeks ago
README.md	initial commit	2 weeks ago
resonate.sql	Reject unawaitable and malformed await requests	4 days ago

README | Code of conduct | Contributing | Apache-2.0 license | Security

Resonate on Postgres

Dead simple durable execution.

About

Resonate Durable Execution in a single SQL file

durable-execution | durable-promises | plpgsql | postgres | postgresql | resonate | sql | supabase | workflow-engine

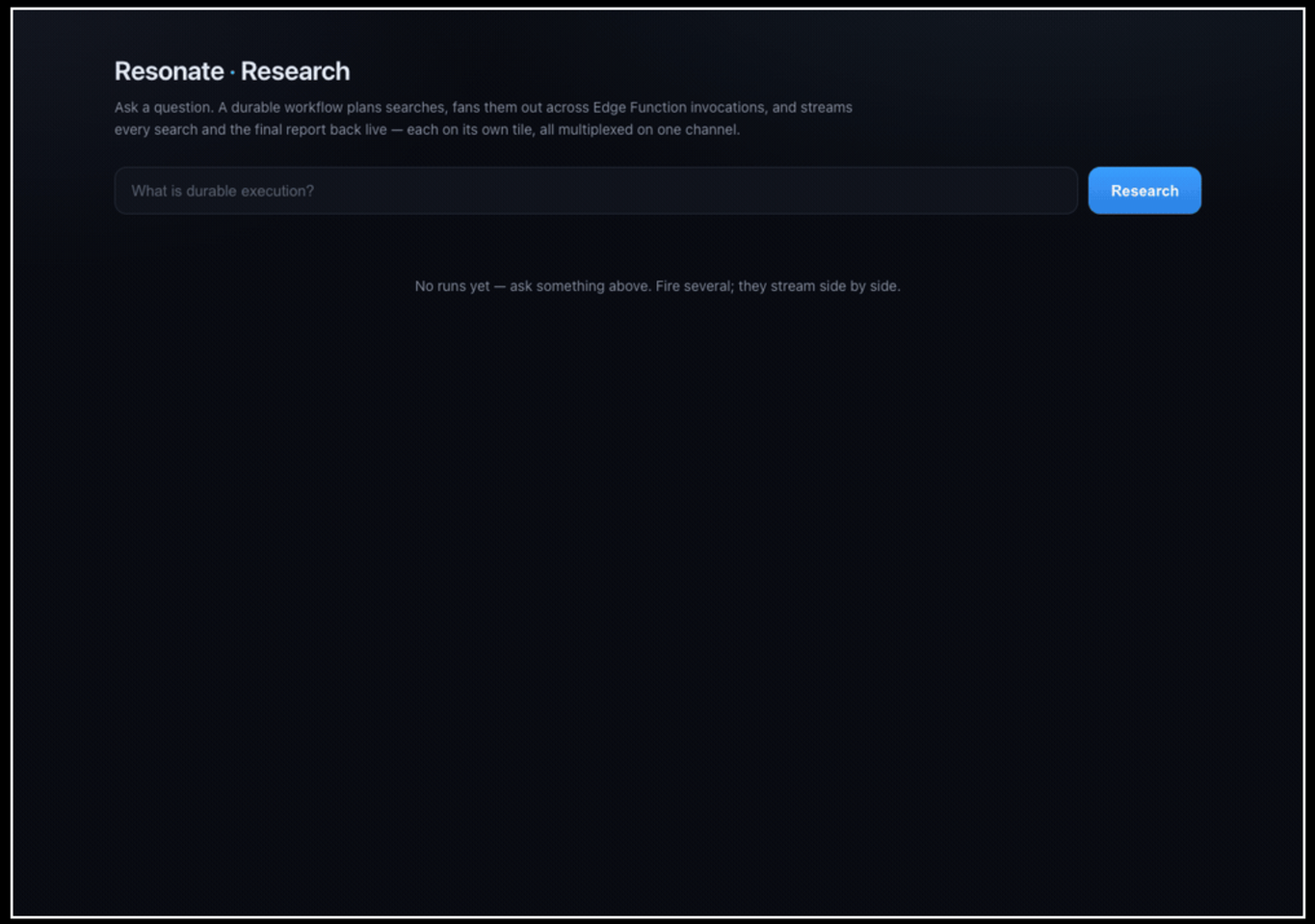
Readme
Apache-2.0 license
Code of conduct
Contributing
Security policy
Activity
Custom properties
9 stars
1 watching
0 forks
Audit log
Report repository

Releases

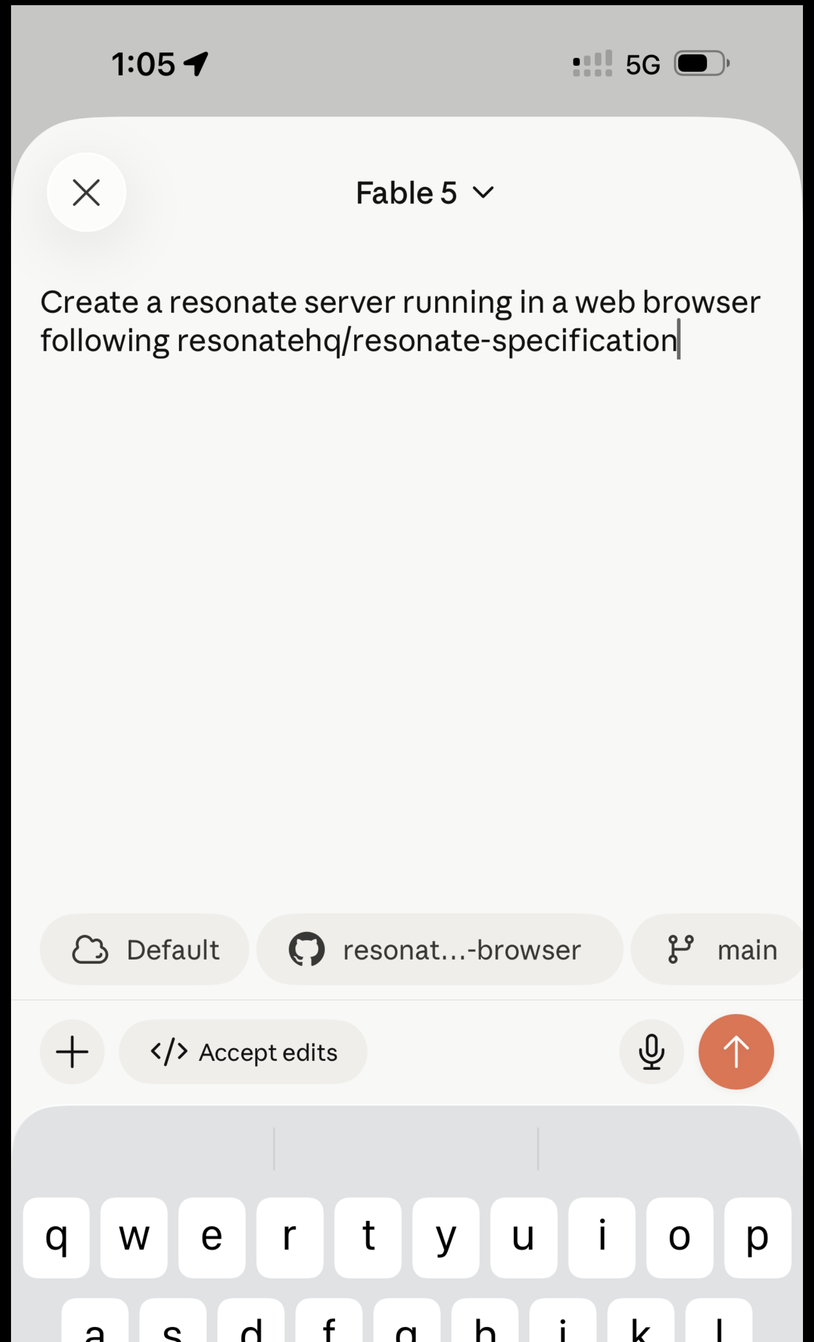
No releases published
Create a new release

Packages

No packages published



https://github.com/resonatehq/resonate-pg


1:05
5G
Fable 5
Create a resonate server running in a web browser following resonatehq/resonate-specification
Default
resonat...-browser
main
Accept edits
q w e r t y u i o p

Distributed durable execution in one tab: editor → Web Worker fleet → IndexedDB

> The Resonate server lives on this page (database resonate-dist, inspect it in DevTools → Application/Storage → IndexedDB). Each worker below is a real **Web Worker** running a full Resonate SDK instance, connected to the server over postMessage. Edit the workflow code, **apply** it to the fleet, and **run**: every ctx.beginRpc call becomes a durable promise routed to *any* worker — and a killed worker's tasks are re-routed to the survivors. Try slowFib with [6], then kill a worker mid-run.

Workflow code (runs inside every worker)

```
1  // Durable, distributed fibonacci.
2  //
3  // Every invocation is a durable promise in IndexedDB. ctx.beginRpc sends
4  // each recursive call to ANY worker in the fleet (anycast), so the
5  // computation spreads across the Web Workers — watch the lanes light up.
6  // A suspended parent may even resume on a different worker than it
7  // started on: the state lives in the database, not in the worker.
8
9  function* fib(ctx, n) {
10   if (n <= 1) return n;
11   const a = yield ctx.beginRpc("fib", n - 1);
12   const b = yield ctx.beginRpc("fib", n - 2);
13   return (yield a) + (yield b);
14 }
15
16 // Slow-motion variant: a durable timer before each step. Start it with a
17 // larger n, then kill a worker mid-run and watch the fleet recover.
18
19 function* slowFib(ctx, n) {
20   yield* ctx.sleep(1000);
21   if (n <= 1) return n;
```

3 workers | apply code & (re)start fleet | run slowFib ([5]) run

Worker fleet

worker 1 w1g1 — 122 — kill

slowFib(3)
slowFib(0)
slowFib(2)
slowFib(3)
slowFib(1)
slowFib(1)
slowFib(2)
slowFib(0)
slowFib(3)
slowFib(1)

~~**worker 2** w2g1 — 110 — kill~~

~~slowFib(1)~~
~~slowFib(3)~~
~~slowFib(3)~~
~~slowFib(5)~~
~~fib(10)~~
~~fib(8)~~
~~fib(8)~~
~~fib(6)~~
~~fib(6)~~
~~fib(6)~~

worker 3 w3g1 — 122 — kill

slowFib(5)
slowFib(4)
slowFib(1)
slowFib(2)
slowFib(0)
slowFib(0)
slowFib(1)
slowFib(0)
slowFib(1)
slowFib(1)

Runs

```
21:22:27 worker 2 killed — its claimed tasks recover via lease timeout
21:22:23 ▶ rpc("slowFib(5)#2-g1") dispatched to the fleet
21:22:23 ✔ fib(10)#1-g1 → 55 [4647 ms]
21:22:19 ▶ rpc("fib(10)#1-g1") dispatched to the fleet
21:22:19 worker 3 ready: fib, slowFib
21:22:19 worker 2 ready: fib, slowFib
21:22:19 worker 1 ready: fib, slowFib
21:22:19 fleet of 3 workers started (generation 1)
```

promises: 207 tasks: 192 promiseTimeouts: 8 taskTimeouts: 0 outbox: 0 deferred: 0

Resonate HQ